Комитет образования и науки Волгоградской области
Государственное бюджетное профессиональное образовательное учреждение
«Волжский политехнический техникум»

КУРСОВОЙ ПРОЕКТ

Дисциплина: МДК 02.02 «Инструментальные средства разработки программного обеспечения»

Тема: «Разработка клиент-серверной системы управления доставкой заказов»

Студент _____ / _____ /

Группа _____

Специальность
_____

Преподаватель _____ / _____ /

Волжский
2026

Утверждаю
Зам. директора по УМР
_____ Е.П. Куневич
«___» _____ 20__ г.

ЗАДАНИЕ
на курсовой проект

По учебной дисциплине: МДК 02.02 «Инструментальные средства разработки программного обеспечения»

Тема: «Разработка клиент-серверной системы управления доставкой заказов»

Специальность: _____

Студенту: _____ Группа: _____

Исходные данные:

1. Техническое задание на систему управления доставкой заказов (роли: клиент, оператор, курьер).
2. Технологии: C#, .NET 9, ASP.NET Core, PostgreSQL, WPF, xUnit, Docker, Git.
3. Требования к оформлению: ГОСТ 2.105-2019, методические указания по оформлению курсового проекта.
4. Репозиторий проекта: https://github.com/Levikaci/order-delivery-managment-system.

Содержание проекта:

Пояснительная записка:

– Введение
– 1 Анализ и проектирование программной системы
– 2 Разработка клиент-серверной программной системы
– 3 Тестирование, отладка и развёртывание
– Заключение
– Список использованных источников
– Приложения

Графическая часть:

1 Архитектурная схема системы.
2 Схема базы данных.
3 Диаграммы вариантов использования и состояний доставки.

Дата выдачи проекта: «___» _____ 20__ г.
Дата сдачи проекта: «___» _____ 20__ г.

Руководитель проекта _____ / _____ /

Председатель ПЦК _____ / _____ /

СОДЕРЖАНИЕ

ВВЕДЕНИЕ....3
1 АНАЛИЗ И ПРОЕКТИРОВАНИЕ ПРОГРАММНОЙ СИСТЕМЫ....5
1.1 Анализ предметной области....5
1.2 Назначение и границы программной системы....6
1.3 Функциональные и нефункциональные требования....7
1.4 Сценарии использования системы....8
1.5 Архитектура клиент-серверного приложения....10
1.6 Модули программной системы и их взаимодействие....11
1.7 Проектирование базы данных....12
1.8 Выбор технологий и инструментальных средств разработки....13
2 РАЗРАБОТКА КЛИЕНТ-СЕРВЕРНОЙ ПРОГРАММНОЙ СИСТЕМЫ....15
2.1 Структура программного проекта....15
2.2 Организация репозитория и процесса разработки....16
2.3 Разработка серверной части....17
2.4 Реализация доступа к данным....19
2.5 Разработка API и контрактов взаимодействия....21
2.6 Разработка клиентской части....23
2.7 Интеграция клиентской и серверной частей....25
2.8 Обработка ошибок и журналирование....26
2.9 Конфигурирование....26
3 ТЕСТИРОВАНИЕ, ОТЛАДКА И РАЗВЁРТЫВАНИЕ....28
3.1 Стратегия тестирования....28
3.2 Модульное тестирование....29
3.3 Интеграционное тестирование....30
3.4 Отладка программной системы....31
3.5 Инспекция и повышение качества программного кода....33
3.6 Контейнеризация программной системы....34
3.7 Развёртывание системы с использованием Docker....35
3.8 Проверка работоспособности развёрнутой системы....36
ЗАКЛЮЧЕНИЕ....38
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ....40
Приложение А – Архитектурная схема программной системы....42
Приложение Б – Схема базы данных....43
Приложение В – Спецификация API....44
Приложение Г – Dockerfile и docker-compose.yml....46
Приложение Д – Дополнительные результаты тестирования....48
Приложение Е – Граф истории Git....49
Приложение Ж – Дополнительные листинги кода....50

ВВЕДЕНИЕ

Доставка заказов остаётся одним из самых массовых и при этом слабо формализованных бизнес-процессов: заказ принимается в одном канале, адрес уточняется в другом, курьер получает задание в мессенджере, а статус выполнения фиксируется «по памяти». Любой сбой в этой цепочке приводит к потерянным заказам, спорам о времени и ответственности и невозможности восстановить, что именно произошло. Современные учебные и производственные проекты решают эту задачу построением клиент-серверных приложений, где единый сервер хранит данные и применяет правила, а клиентские приложения обеспечивают работу каждой роли [11, 13].

Актуальность темы определяется тем, что на примере небольшой системы доставки можно пройти полный цикл разработки программного продукта: от анализа предметной области и проектирования архитектуры до тестирования, контейнеризации и развёртывания. Такая система содержит типовые для корпоративных приложений элементы: ролевую модель доступа, жизненный цикл сущностей со строгими правилами переходов, историю изменений, поиск, справочники и единый формат ошибок.

Объектом разработки является программная система управления доставкой заказов, **предметом** – архитектура, программная реализация, тестирование и развёртывание клиент-серверного приложения на платформе .NET.

Цель работы – разработать, протестировать и подготовить к развёртыванию клиент-серверную программную систему, которая автоматизирует приём заказов, назначение курьеров, контроль статусов доставки и ведение истории изменений.

Для достижения цели необходимо решить следующие **задачи**:

– проанализировать предметную область, определить границы системы и сформулировать функциональные и нефункциональные требования;

– спроектировать модульную архитектуру, контракты взаимодействия модулей и схему базы данных;

– реализовать серверную часть на ASP.NET Core с хранением данных в PostgreSQL и REST API;

– реализовать клиентское приложение на WPF с интерфейсами для ролей клиента, оператора и курьера;

– разработать автоматические модульные и интеграционные тесты, провести отладку и инспекцию кода;

– подготовить контейнеризацию и описать развёртывание системы с помощью Docker.

Методы разработки. При выполнении работы использовались объектно-ориентированное проектирование, модульный подход с общением через контракты, тестирование на реальной базе данных, система контроля версий Git и контейнеризация [8, 10].

Практическая значимость заключается в том, что результат представляет собой работоспособное приложение с открытым исходным кодом, которое запускается одной командой, покрыто 277 автоматическими тестами и сопровождается документацией: описанием архитектуры, контрактов, решений по проекту и отчётами по этапам курсового проектирования.

Пояснительная записка состоит из введения, трёх разделов, заключения, списка использованных источников и семи приложений. В первом разделе выполнены анализ и проектирование системы, во втором описана её разработка, в третьем – тестирование, отладка и развёртывание. Исходный код размещён в репозитории `Levikaci/order-delivery-managment-system` на GitHub. Записка оформлена в соответствии с ГОСТ 2.105-2019 [1] и ГОСТ 2.104-2006 [2].

1 АНАЛИЗ И ПРОЕКТИРОВАНИЕ ПРОГРАММНОЙ СИСТЕМЫ

1.1 Анализ предметной области

Под доставкой заказа понимается совокупность действий от момента, когда клиент оформил заказ, до момента, когда он получил товар и подтвердил получение. В процессе участвуют три стороны: клиент, который выбирает товары и указывает адрес; оператор (диспетчер), который контролирует выполнение и распределяет заказы между курьерами; курьер, который физически выполняет доставку и сообщает о её ходе.

Основные проблемы, возникающие при ручной организации процесса (в таблицах, мессенджерах и по телефону):

– отсутствие единого источника правды о статусе заказа: клиент, оператор и курьер видят разную информацию;

– невозможность гарантировать, что курьера не назначат на два заказа одновременно;

– отсутствие истории статусов, из-за чего нельзя установить время начала и завершения доставки и причину неудачи;

– ошибки при передаче адреса и состава заказа между участниками;

– отсутствие разграничения прав: любой участник может изменить любые данные.

Универсальные CRM-системы решают часть этих задач, однако их внедрение проходит несколько этапов – от планирования и интеграции до тестирования, оптимизации и сопровождения – и требует значительных ресурсов [13]. Для небольшой службы доставки оправдан компактный специализированный продукт, который выполняет только необходимые операции. Сравнение подходов приведено в таблице 1.

Таблица 1 – Сравнение подходов к учёту доставки

Критерий	**Ручной учёт (таблицы, мессенджеры)**	**Универсальная CRM**	**Разработанная система**
Единый статус заказа	Нет	Да	Да
Контроль занятости курьера	Вручную	Требует настройки	Автоматически (состояния курьера)
История статусов и время доставки	Нет	Частично	Да, для доставки и заказа

Критерий	Ручной учёт (таблицы, мессенджеры)	Универсальная CRM	Разработанная система
Разграничение ролей	Нет	Да	Клиент, оператор, курьер
Сложность внедрения	Минимальная	Высокая	Запуск одной командой

Таким образом, предметная область характеризуется небольшим числом сущностей (клиент, адрес, товар, заказ, доставка, курьер), строгим жизненным циклом доставки и необходимостью ролевого доступа. Эти особенности определили выбор клиент-серверной архитектуры с единым сервером бизнес-правил.

1.2 Назначение и границы программной системы

Система предназначена для автоматизации управления доставкой заказов. Она обеспечивает регистрацию заказов, хранение данных клиентов и их адресов, назначение курьеров, отслеживание статусов, ведение истории изменений и фиксацию результата доставки. Технические условия на систему были сформулированы в виде технического задания по ГОСТ 34.602-2020 [3] и затем уточнялись в ходе этапов курсового проектирования.

Состав функций, входящих в систему и не входящих в неё, показан в таблице 2.

Таблица 2 – Границы программной системы

Входит в систему	Не входит в систему
Заказы с составом (товары, количество) и итоговой суммой	Реальные банковские платежи и интеграции
Клиенты и их сохранённые адреса	GPS-навигация и построение маршрутов
Доставки, статусы и история статусов	Внешние картографические сервисы
Курьеры, их состояния и контроль доступности	Реальная SMS-интеграция
Поиск и просмотр заказов, справочники	Мобильное приложение
Вход по логину и паролю, три роли	Интеграция с маркетплейсами

Граница системы проходит по REST API: всё, что находится за его пределами (клиентские приложения, внешние сервисы), взаимодействует с сервером только через HTTP и формат JSON. Это позволяет заменять клиента без изменения серверной части – именно так было сделано при переходе с веб-интерфейса на WPF-приложение.

1.3 Функциональные и нефункциональные требования

Функциональные требования сформулированы по ролям пользователей и пронумерованы для последующей трассировки на реализацию и тесты (таблица 3).

Таблица 3 – Функциональные требования

Код	Требование	Роль
ФТ-1	Вход в систему по логину и паролю, получение токена доступа	Все
ФТ-2	Создание заказа из товаров с указанием количества и адреса доставки	Клиент
ФТ-3	Выбор сохранённого адреса или ввод нового с возможностью сохранения	Клиент
ФТ-4	Изменение и отмена заказа, пока курьер не принял его в работу	Клиент
ФТ-5	Автоматическое создание доставки и подбор свободного курьера при создании заказа	Система
ФТ-6	Назначение курьера вручную или автоматически, повтор при отсутствии свободных	Оператор
ФТ-7	Смена статуса доставки по допустимым переходам, причина при неудаче	Оператор, курьер
ФТ-8	Фиксация времени начала и завершения доставки, история статусов доставки и заказа	Система
ФТ-9	Поиск и фильтрация доставок по тексту, статусу, курьеру, клиенту	Оператор
ФТ-10	Ведение справочников клиентов, курьеров, товаров и адресов	Оператор
ФТ-11	Подтверждение получения заказа после статуса «Доставлено»	Клиент
ФТ-12	Переключение курьера между состояниями «на линии» и «офлайн»	Курьер

Нефункциональные требования определяют качество реализации и условия эксплуатации. Они приведены в таблице 4.

Таблица 4 – Нефункциональные требования

Код	Требование	Способ обеспечения
НТ-1	Единый формат ошибок со стабильными кодами	Метод Err в ApiHost, коды в таблице сообщений
НТ-2	Целостность данных при составных операциях	Транзакции PostgreSQL, внешние ключи, уникальные индексы
НТ-3	Безопасное хранение паролей	PBKDF2-SHA256, соль, 100 000 итераций

Код	Требование	Способ обеспечения
НТ-4	Проверка прав на каждой операции	Фильтр ролей конечных точек
НТ-5	Воспроизводимый запуск	Dockerfile, docker-compose, скрипты run
НТ-6	Проверяемость	277 автоматических тестов, предупреждений компилятора нет
НТ-7	Журналирование ключевых событий	ILogger, структурные параметры

1.4 Сценарии использования системы

Для описания взаимодействия пользователей с системой построена диаграмма вариантов использования (рисунок 1). На ней показаны три актёра и функции, доступные каждому из них. Вход в систему и просмотр состояния являются общими, остальные варианты ограничены ролью.



Рисунок 1 – Диаграмма вариантов использования

На основе технического задания определены три сквозных сценария, которые затем были закреплены интеграционными тестами.

Сценарий 1. Успешное создание заказа. Клиент отправляет `POST /api/orders` с составом и адресом. Сервер проверяет данные, в одной транзакции создаёт заказ, его позиции и доставку, записывает первую запись истории и подбирает свободного курьера. Курьер переходит в состояние «Занят», доставка получает статус `Assigned`, заказ – `Confirmed`. Последовательность взаимодействия показана на рисунке 2.

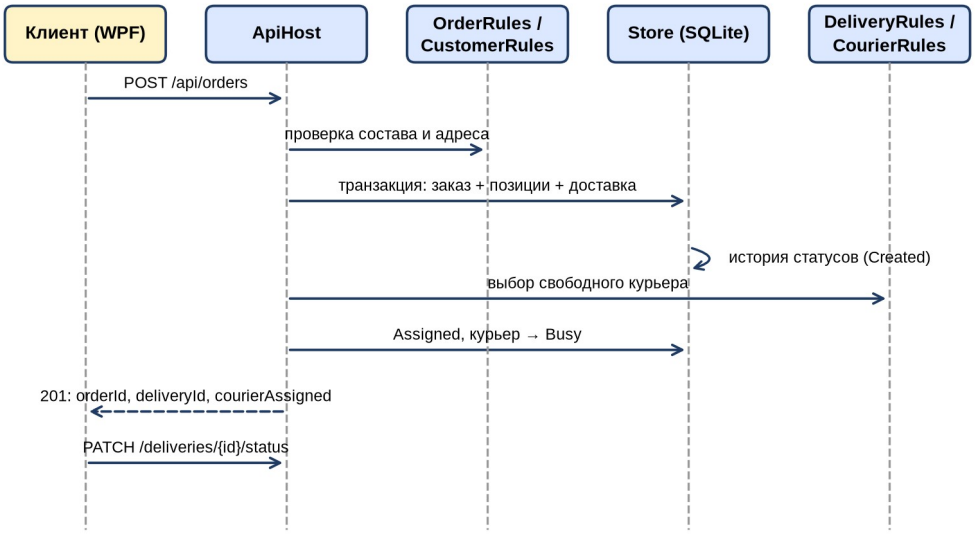


Рисунок 2 – Диаграмма последовательности создания заказа

Сценарий 2. Нет свободных курьеров. Заказ и доставка сохраняются, но доставка остаётся в статусе `Created`. Ответ содержит признак `courierAssigned = false` и подсказку, как повторить назначение. Оператор может назначить курьера позже вручную или автоматически.

Сценарий 3. Полный жизненный цикл. Курьер последовательно принимает заказ, начинает движение и подтверждает доставку. На каждом шаге сервер проверяет допустимость перехода, фиксирует время и запись истории; после финального статуса курьер освобождается, а клиент может подтвердить получение.

Статусы доставки образуют конечный автомат, представленный на рисунке 3. Заказ имеет отдельный набор статусов, который выводится из статуса доставки по правилу, описанному в разделе 2.3.

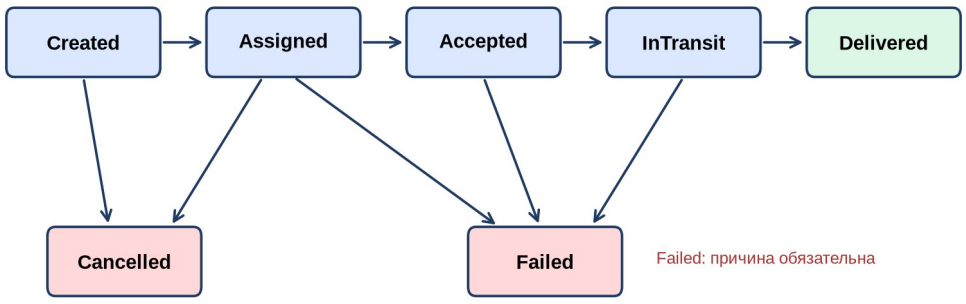


Рисунок 3 – Диаграмма состояний доставки

1.5 Архитектура клиент-серверного приложения

Система построена по клиент-серверной архитектуре с «толстым» клиентом. Серверная часть (`ApiHost`) реализует всю бизнес-логику и хранение данных и предоставляет REST API с обменом данными в формате JSON. Клиентская часть (`WpfClient`) отвечает только за представление и отправку запросов. Такое разделение позволяет независимо развивать и тестировать каждую часть, а также разворачивать сервер в контейнере, не затрагивая клиента [11].

Общая структура системы показана на рисунке 4 (укрупнённая схема – в приложении А). Клиент обращается к серверу по протоколу HTTP, передавая токен доступа в заголовке `Authorization`. Внутри сервера выделены три слоя: слой маршрутизации и проверки ролей (`Program.cs`), слой сессий (`Auth.cs`) и слой сценариев и доступа к данным (`Store.cs`). Бизнес-правила вынесены в четыре независимых модуля, которые зависят только от проекта общих контрактов.

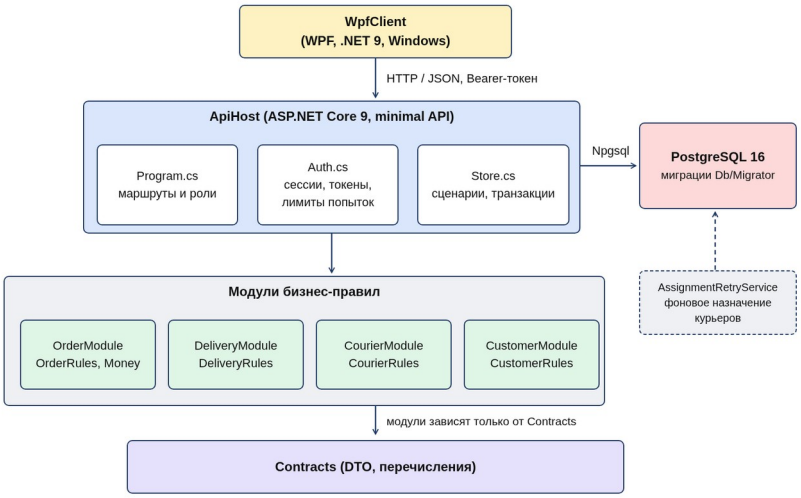


Рисунок 4 – Архитектура программной системы

Выбор «монолита с модулями», а не набора микросервисов, обоснован масштабом задачи: один процесс, одна база данных и транзакционные операции, затрагивающие заказы, доставки и курьеров одновременно. Границы модулей при этом выдерживаются на уровне проектов решения, что оставляет возможность выделить модуль в отдельный сервис без переписывания правил. Это решение зафиксировано в документе ADR-001.

1.6 Модули программной системы и их взаимодействие

Решение разделено на проекты, каждый из которых отвечает за свою область (таблица 5). Модули бизнес-правил реализованы в виде статических классов с чистыми функциями без побочных эффектов, что упрощает их тестирование.

Таблица 5 – Модули системы

Проект	Ответственность	Ключевые элементы
Contracts	Общие DTO, перечисления и интерфейсы	DeliveryStatus, OrderStatus, CourierState, IDeliveryService
OrderModule	Правила заказа: состав, редактирование, отмена	OrderRules, OrderService
DeliveryModule	Правила доставки: переходы статусов, связь со статусом заказа	DeliveryRules, DeliveryService
CourierModule	Доступность курьеров и смена состояния	CourierRules
CustomerModule	Проверка данных клиента и адреса, форматирование адреса	CustomerRules
ApiHost	HTTP API, роли, хранилище PostgreSQL, фоновое назначение	Program, Auth, Store, Domain
WpfClient	Клиентское приложение для трёх ролей	LoginWindow, MainWindow, Api

Взаимодействие модулей организовано через контракты (ADR-002): модуль заказов не знает о конкретной реализации доставки и обращается к ней через интерфейс `IDeliveryService`, объявленный в проекте `Contracts`. Прямые ссылки между модулями допускаются только там, где одно правило является прямым следствием другого – например, `OrderRules` использует `DeliveryRules.ToOrderStatus` для вывода статуса заказа из статуса доставки. Сценарии, затрагивающие данные нескольких модулей, выполняются в `Store` в рамках одной транзакции и применяют правила модулей к данным.

Для проверки допустимости вызовов определён набор протоколов взаимодействия, описанных в документации проекта: протокол создания доставки для заказа (повторный вызов с тем же заказом возвращает ошибку `DELIVERY_ALREADY_EXISTS`) и протокол назначения курьера (свободный курьер выбирается только из состояния `Available`). Подробно контракты рассмотрены в разделе 2.5.

1.7 Проектирование базы данных

Для хранения данных выбрана серверная СУБД PostgreSQL 16: она поддерживает транзакции, внешние ключи, частичные уникальные индексы и параллельную работу нескольких подключений, а значит, подходит и для рабочего развёртывания, где к серверу одновременно обращаются несколько экземпляров приложения [7]. Для локальной разработки база запускается одной командой docker compose up db, данные хранятся в именованном томе pgdata.

Схема базы данных состоит из четырнадцати таблиц, включая служебную schema_migrations (рисунок 5, увеличенная схема – в приложении Б). Назначение таблиц приведено в таблице 6.

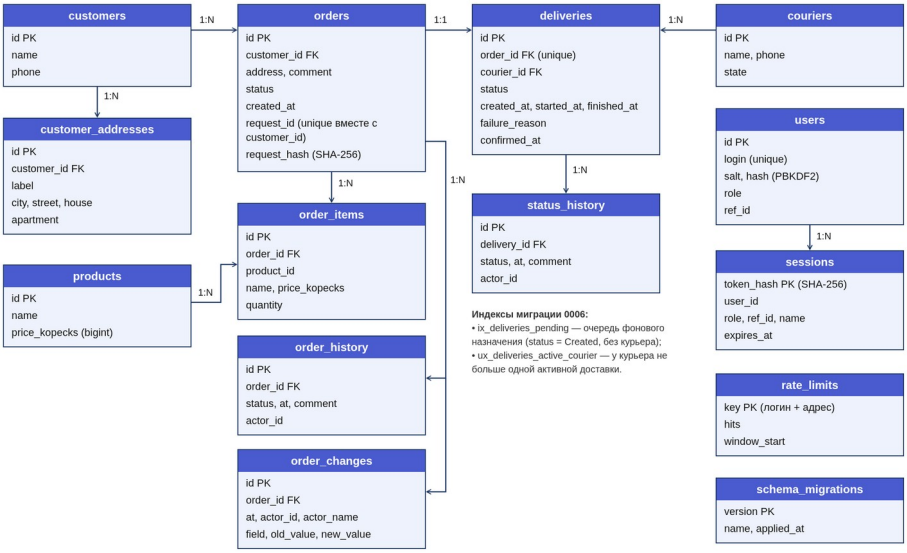


Рисунок 5 – Схема базы данных

Таблица 6 – Таблицы базы данных

Таблица	Назначение	Особенности
customers	Клиенты	Имя и телефон
customer_addresses	Сохранённые адреса клиента	Каскадное удаление при удалении клиента
users	Учётные записи	Уникальный логин, соль и хэш пароля, роль, ссылка на клиента или курьера
couriers	Курьеры	Состояние: Available, Busy, Offline, Blocked
products	Справочник товаров	Название и цена
orders	Заказы	Адрес копируется в заказ, статус заказа
order_items	Позиции заказа	Название и цена товара фиксируются в позиции

Таблица	Назначение	Особенности
deliveries	Доставки	Уникальный заказ, курьер, время начала и завершения, причина неудачи
status_history	История статусов доставки	Статус, время, комментарий
order_history	История статусов заказа	Статус, время, комментарий
order_changes	Журнал изменений заказа	Старое и новое значение, автор, время
sessions	Сессии	Хэш токена, пользователь, срок действия
rate_limits	Счётчики попыток	Входы и регистрации по ключу «логин + адрес»
schema_migrations	Версии миграций	Номер и время применения

Ключевые проектные решения по базе данных:

– цена и название товара копируются в позицию заказа, поэтому изменение справочника не меняет сумму уже оформленных заказов;

– связь «один заказ – одна доставка» обеспечена ограничением UNIQUE на deliveries.order_id, а правило «у курьера не больше одной активной доставки» – частичным уникальным индексом, поэтому создание доставки идемпотентно, а гонка двух назначений заканчивается отказом базы, а не двойным назначением;

– пароли хранятся в виде хэша PBKDF2 с индивидуальной солью, открытый пароль не сохраняется;

– схема изменяется пронумерованными миграциями (таблица schema_migrations): каждая выполняется в транзакции, при запуске применяются только недостающие, а база, созданная более новой версией программы, отклоняется;

– для частых выборок созданы индексы по статусу и курьеру доставки, клиенту заказа и внешним ключам таблиц истории.

1.8 Выбор технологий и инструментальных средств разработки

Технологический стек выбирался исходя из требований к кроссплатформенности сервера, скорости разработки, наличию средств тестирования и воспроизводимости развёртывания. Выбор представлен в таблице 7.

Таблица 7 – Технологии и инструменты

Область	Выбор	Обоснование
Язык и платформа	C# 12, .NET 9	Строгая типизация, единая платформа для сервера, клиента и тестов [4]
Сервер	ASP.NET Core 9, minimal API	Высокая производительность, компактное описание конечных точек [5, 11]

Область	Выбор	Обоснование
Хранение данных	PostgreSQL 16, Npgsql	Транзакции, параллельный доступ, миграции, параметризованные запросы [7]
Клиент	WPF (.NET 9)	Полноценное настольное приложение без сторонних пакетов [6]
Тестирование	xUnit	Теории с параметрами, изоляция тестов [9, 14]
Развёртывание	Docker, docker-compose	Контейнеры app и db, том для данных, отдельный файл для сервера [8, 12, 16]
Контроль версий	Git, GitHub	История изменений, ветки, обзор кода [10]

Платформа ASP.NET Core выбрана в том числе потому, что её применение для веб-приложений с CRUD-операциями и REST-интерфейсом, включая нагрузочное тестирование, подробно описано в [11], а минимальный API позволяет описать конечные точки и фильтры ролей в одном файле без лишней церемонии. WPF выбран как клиент, потому что сервер и клиент разрабатываются на одном языке, а общий проект контрактов исключает расхождение форматов данных. Контейнеризация упрощает воспроизводимость запуска и сокращает время подготовки окружения [12, 16].

2 РАЗРАБОТКА КЛИЕНТ-СЕРВЕРНОЙ ПРОГРАММНОЙ СИСТЕМЫ

2.1 Структура программного проекта

Программная система оформлена как одно решение Visual Studio `OrderDeliveryManagementSystem.sln`, в которое входят шесть проектов исходного кода и два тестовых проекта. Такое деление повторяет модульную архитектуру, спроектированную в подразделе 1.6: каждая предметная область вынесена в отдельную библиотеку классов, а серверный проект и клиентское приложение зависят только от контрактов и правил этих библиотек [4, 5].

Состав решения, назначение проектов и их размер в строках исходного кода приведены в таблице 8. Подсчёт выполнен по файлам `*.cs` без учёта автоматически генерируемых файлов и разметки XAML. Дополнительные листинги кода приведены в приложении Ж.

Таблица 8 – Структура решения и размер проектов

Проект	Тип	Назначение	Строк кода
`src/ApiHost`	ASP.NET Core Minimal API	Сервер: REST API, аутентификация, хранилище PostgreSQL	1901
`src/Contracts`	Библиотека классов	Перечисления статусов, DTO и интерфейсы модулей	10
`src/OrderModule`	Библиотека классов	Правила и сервис заказов	54
`src/ DeliveryModule`	Библиотека классов	Переходы статусов доставки, сервис доставки	48
`src/CourierModule`	Библиотека классов	Правила назначения и состояния курьеров	19
`src/ CustomerModule`	Библиотека классов	Проверка имени и адреса клиента	35
`src/WpfClient`	WPF (net9.0-windows)	Клиентское приложение для трёх ролей	1118
`tests/ OrderModule.Tests`	xUnit	Модульные тесты правил и сервисов	—
`tests/ Integration.Tests`	xUnit	Интеграционные тесты на реальной PostgreSQL	—

Суммарно исходный код составляет около 5 900 строк на C#, из них 3 185 строк приходится на программу и около 2 800 строк – на автоматические тесты.

Все функции технического задания реализованы, а объём тестов сопоставим с объёмом программы.

Проекты связаны однонаправленно: `ApiHost` ссылается на `Contracts` и все модули правил, модули ссылаются только на `Contracts`, клиент `WpfClient` не ссылается ни на один серверный проект и общается с сервером исключительно по HTTP. Благодаря этому клиент можно заменить (например, веб-интерфейсом или мобильным приложением), не изменяя сервер, а правила предметной области можно проверять тестами без запуска веб-сервера и базы данных.

2.2 Организация репозитория и процесса разработки

Исходный код хранится в системе контроля версий Git [10] в репозитории на GitHub. Основной веткой является main; изменения вносились небольшими законченными коммитами с осмысленными сообщениями на русском языке. В приложении Е приведены пять последних коммитов основной ветки: исправления безопасности и надёжности (идемпотентность создания заказа, фоновое назначение курьеров, хранение сессий и счётчиков попыток в PostgreSQL), проверка входных данных, доработка клиента WPF и обновление документации. Изменения вносились через запрос на слияние с описанием проверок и известных ограничений.

Состав репозитория поддерживает воспроизводимую сборку: в корне находятся файл решения, каталоги `src` и `tests`, каталог документации `docs`, файлы `Dockerfile` и `docker-compose.yml`, скрипты запуска `run.bat`, `run.sh` и `run-client.bat`, а также файл `.editorconfig` с правилами форматирования. Проектная документация размещена в каталоге `docs`: архитектура, контракты и модули, описание зависимостей, стратегия тестирования, матрица прослеживаемости требований, отчёты и чек-листы по этапам, три записи архитектурных решений (ADR) – о модульной архитектуре, контрактах модулей и истории статусов.

Процесс разработки был итеративным и соответствовал пяти этапам курсового проекта. На первом этапе зафиксированы границы системы и требования, на втором – архитектура и модульные контракты, на третьем – реализация серверной и клиентской частей, на четвёртом – тестирование и отладка, на пятом – инспекция кода и контейнеризация. Результаты каждого этапа оформлялись отчётом и чек-листом в каталоге `docs`, что позволяло сверять сделанное с требованиями на любом шаге.

Единый стиль кода обеспечивается файлом `.editorconfig`: отступы в четыре пробела, кодировка UTF-8, переводы строк CRLF для файлов C#, XAML и проектов. Соответствие проверяется командой `dotnet format whitespace --`

`verify-no-changes`; перед публикацией форматирование применено ко всем файлам решения, и повторная проверка завершается без замечаний.

2.3 Разработка серверной части

Серверная часть реализована в проекте `ApiHost` на ASP.NET Core 9 с использованием Minimal API [5, 11]. Точка входа – файл `Program.cs`. В нём создаётся приложение, регистрируются два сервиса-одиночки (`Store` – хранилище данных и бизнес-операции, `AuthService` – сессии пользователей), задаются настройки сериализации JSON (перечисления передаются строками, что делает API читаемым) и описываются все конечные точки.

Адрес прослушивания определяется переменными окружения и аргументами командной строки. Если он не задан ни одним из способов (`--urls`, `ASPNETCORE_URLS`, `http_ports`, `launchSettings.json`), сервер слушает порт 8080 – тот же, что используется клиентом по умолчанию, в Dockerfile и документации. Такое решение устраняет типичную ошибку, при которой клиент и сервер запускаются на разных портах (подраздел 3.4).

Аутентификация построена на токенах. После проверки логина и пароля класс AuthService создаёт случайный токен длиной 24 байта (48 шестнадцатеричных символов), сохраняет сессию в таблице sessions со сроком жизни 12 часов и возвращает токен клиенту; в базе хранится только хэш SHA-256 от токена, поэтому утечка таблицы не раскрывает действующих токенов, а сессии переживают перезапуск сервера. Клиент передаёт токен в заголовке Authorization: Bearer …. Сессия содержит роль пользователя (client, operator, courier) и идентификатор связанной сущности – клиента или курьера; при удалении пользователя его сессии отзываются. Пароли в базе данных не хранятся в открытом виде: используется алгоритм PBKDF2 с хэш-функцией SHA-256, 100 000 итераций и случайной солью длиной 16 байт. Подбор пароля ограничен: после пяти неудачных попыток для пары «логин и адрес клиента» вход блокируется на пять минут; счётчики хранятся в таблице rate_limits и поэтому работают при нескольких экземплярах сервера. Самостоятельная регистрация клиентов (POST /api/auth/register) ограничена по числу попыток с одного адреса и может быть отключена настройкой.

Авторизация реализована фильтром конечных точек `Roles(…)`. Он извлекает сессию из заголовка запроса, при её отсутствии возвращает ошибку `UNAUTHORIZED` (HTTP 401), а при несоответствии роли – `FORBIDDEN` (HTTP 403). Фильтр применяется декларативно при описании маршрута, поэтому правила доступа видны рядом с обработчиком и не размазаны по коду (листинг 1).

Листинг 1 – Формирование ответа об ошибке и фильтр доступа по ролям (src/ApiHost/Program.cs)

```
IResult Err(string code)
{
    var status = code switch
    {
        "UNAUTHORIZED" or "INVALID_CREDENTIALS" => 401,
        "FORBIDDEN" or "UNAUTHORIZED_STATUS_CHANGE" => 403,
        _ when code.EndsWith("NOT_FOUND") => 404,
        _ when code.EndsWith("HAS_ORDERS") || code.StartsWith("DELIVERY_ALREADY")
            || code is "COURIER_NOT_AVAILABLE" or "COURIER_BUSY"
            or "INVALID_STATUS_TRANSITION" or "ORDER_NOT_EDITABLE"
            or "LOGIN_TAKEN" => 409,
        _ => 400
    };
    return Results.Json(new { success = false, errorCode = code, error = code,
        message = messages.GetValueOrDefault(code, code) }, statusCode: status);
}


RouteHandlerBuilder Roles(RouteHandlerBuilder b, params string[] roles) =>
  b.AddEndpointFilter(async (ctx, next) =>
  {
    var s = ctx.HttpContext.RequestServices.GetRequiredService<AuthService>()
              .Get(ctx.HttpContext.Request);
    if (s is null) return Err("UNAUTHORIZED");
    if (roles.Length > 0 && !roles.Contains(s.Role)) return Err("FORBIDDEN");
    ctx.HttpContext.Items["me"] = s;
    return await next(ctx);
  });
```

Бизнес-правила вынесены из обработчиков HTTP в статические классы модулей. Например, класс `DeliveryRules` хранит таблицу допустимых переходов статусов доставки и функцию соответствия статуса доставки статусу заказа (листинг 2). Эта логика не зависит ни от веб-сервера, ни от базы данных, поэтому она покрыта быстрыми модульными тестами (подраздел 3.2).

Листинг 2 – Правила переходов статусов доставки (src/DeliveryModule/DeliveryRules.cs)

```
private static readonly Dictionary<DeliveryStatus, DeliveryStatus[]> Transitions =
    new()
{
    [DeliveryStatus.Created]   = new[] { DeliveryStatus.Assigned,
        DeliveryStatus.Cancelled },
    [DeliveryStatus.Assigned]  = new[] { DeliveryStatus.Accepted,
        DeliveryStatus.Failed,
                                         DeliveryStatus.Cancelled },
    [DeliveryStatus.Accepted]  = new[] { DeliveryStatus.InTransit,
        DeliveryStatus.Failed },
    [DeliveryStatus.InTransit] = new[] { DeliveryStatus.Delivered,
        DeliveryStatus.Failed },
};


public static bool CanTransition(DeliveryStatus from, DeliveryStatus to) =>
    Transitions.TryGetValue(from, out var next) && next.Contains(to);


public static OrderStatus ToOrderStatus(DeliveryStatus s) => s switch
{
    DeliveryStatus.Created   => OrderStatus.Created,
    DeliveryStatus.Assigned  => OrderStatus.Confirmed,
    DeliveryStatus.Accepted  => OrderStatus.Preparing,
    DeliveryStatus.InTransit => OrderStatus.InDelivery,
```

```
    DeliveryStatus.Delivered => OrderStatus.Delivered,
    _                        => OrderStatus.Cancelled,
};
```

Остальные модули устроены аналогично. `OrderRules` проверяет состав заказа (код `EMPTY_ORDER`, если позиций нет, и `INVALID_QUANTITY`, если количество вне диапазона от 1 до 99), определяет, когда заказ можно изменить или отменить клиентом (только в статусах «Создана» и «Назначена»), и считает итоговую сумму. `CourierRules` решает, можно ли назначить курьера, и проверяет смену его состояния: состояние «Занят» устанавливается только системой при назначении доставки. `CustomerRules` проверяет длину имени (до 100 символов) и обязательные поля адреса, а также формирует строку адреса вида «г. …, ул. …, д. …, кв. …».

Для демонстрации принципа работы через контракты в модуле доставки реализован сервис `DeliveryService` с интерфейсом `IDeliveryService` из проекта `Contracts`. Метод `CreateAsync` проверяет входные данные, отклоняет повторное создание доставки с тем же идентификатором и пишет события в журнал с областью (`BeginScope`), в которую включены идентификаторы доставки и клиента. Аналогично устроен `OrderService`, создающий заказ через контракт доставки.

2.4 Реализация доступа к данным

Данные хранятся в PostgreSQL [7] через драйвер Npgsql. Все операции с базой сосредоточены в классе Store, который скрывает от остальной программы SQL-запросы и детали схемы. Выбор PostgreSQL обоснован в подразделе 1.8: сервер поддерживает параллельную работу и частичные уникальные индексы, которые защищают инварианты даже при одновременных запросах.

Схема версионируется пронумерованными миграциями (таблица schema_migrations, на момент написания – седьмая версия). При запуске Store применяет недостающие миграции, каждую в отдельной транзакции, под рекомендательной блокировкой, чтобы одновременный старт нескольких экземпляров не приводил к конфликту. Миграция 0006 перед созданием уникального индекса возвращает в очередь лишние активные доставки курьера, а миграция 0007 добавляет таблицу rate_limits. В режиме разработки пустая база заполняется демонстрационными данными: оператором, двумя клиентами, тремя курьерами и товарами; пароль каждого пользователя совпадает с его логином. В рабочем режиме (ASPNETCORE_ENVIRONMENT=Production) демонстрационные данные запрещены: сервер не стартует со слабым паролем базы или оператора и перечисляет все найденные проблемы сразу.

Потокобезопасность обеспечивается транзакциями PostgreSQL и пулом подключений Npgsql: каждая операция берёт подключение из пула и выполняет работу в транзакции, а гонки (одновременное назначение курьера, повтор запроса с тем же requestId) разрешаются ограничениями базы и блокировками строк. Повтор создания заказа с тем же requestId возвращает тот же заказ, а с другими данными – ошибку 409; фоновая служба AssignmentRetryService с нарастающей паузой повторяет назначение курьеров, если свободных не было или база была недоступна. Все запросы параметризованы ($id, $status и т. п.), конкатенация пользовательских значений в SQL не используется, что исключает SQL-инъекции [15]; спецсимволы % и _ в поиске экранируются.

Операции, меняющие несколько таблиц, выполняются в транзакциях. Создание заказа (листинг 3) в одной транзакции добавляет запись заказа, его позиции, запись истории заказа, запись доставки с историей её статусов и, если клиент попросил, сохраняет новый адрес. Только после фиксации транзакции выполняется автоматическое назначение свободного курьера – отдельной транзакцией. Поэтому сбой при назначении не приводит к потере заказа: он остаётся в статусе «Создана», и назначение можно повторить.

Листинг 3 – Создание заказа в транзакции (фрагмент Store.CreateOrder)

```
var orderId = Guid.NewGuid(); id = Guid.NewGuid();
using var tx = c.BeginTransaction();
Cmd(c, "INSERT INTO orders(id,customer_id,address,comment,status,created_at) " +
       "VALUES($i,$c,$a,$m,$s,$t)", ...).ExecuteNonQuery();
InsertItems(c, orderId, lines);
OrderHist(c, orderId, OrderStatus.Created, "Заказ создан");
InsertDelivery(c, id, orderId);
if (r.SaveAddress && inline is not null)
    AddAddr(c, r.CustomerId, "", inline.City, inline.Street, inline.House,
        inline.Apartment);
tx.Commit();
_log.LogInformation("Создан заказ {Order}, доставка {Delivery}", orderId, id);
...
var assigned = TryAutoAssign(id);
return (Delivery(id), assigned, null);
```

Изменение статуса доставки (метод `ChangeStatus`, листинг 4) – самая насыщенная правилами операция. Сначала проверяются права: клиент может лишь отменить собственную доставку, курьер – менять статусы только своей доставки и только в допустимые для курьера состояния. Затем обрабатывается идемпотентность (повторное применение того же статуса не считается ошибкой), проверяется, не завершена ли доставка, допустим ли переход, и указана ли причина при неудачной доставке. Обновление статуса, запись в историю, изменение статуса заказа и освобождение курьера выполняются одной транзакцией, поэтому данные всегда остаются согласованными.

Листинг 4 – Проверки и обновление при смене статуса (фрагмент Store.ChangeStatus)

```
if (actor.Role == "client" &&
    (actor.RefId != d.CustomerId || s != DeliveryStatus.Cancelled))
        return "UNAUTHORIZED_STATUS_CHANGE";
if (d.Status == s) return null;            // повтор перехода — без изменений
if (DeliveryRules.IsFinal(d.Status)) return "DELIVERY_ALREADY_COMPLETED";
if (!DeliveryRules.CanTransition(d.Status, s)) return "INVALID_STATUS_TRANSITION";
if (s == DeliveryStatus.Failed && string.IsNullOrWhiteSpace(comment))
    return "FAILURE_REASON_REQUIRED";

using var tx = c.BeginTransaction();
Cmd(c, @"UPDATE deliveries SET status=$s,
           started_at=CASE WHEN $s='InTransit' AND started_at IS NULL THEN $t ELSE
               started_at END,
           finished_at=CASE WHEN $f=1 THEN $t ELSE finished_at END
         WHERE id=$i", ...).ExecuteNonQuery();
Hist(c, id, s, comment);
SetOrderStatus(c, d.OrderId, d.OrderStatus, DeliveryRules.ToOrderStatus(s), ...);
if (final && d.CourierId is Guid cid)      // освободить курьера
    Cmd(c, "UPDATE couriers SET state='Available' WHERE id=$i AND state='Busy'",
        ...);
tx.Commit();
```

2.5 Разработка API и контрактов взаимодействия

Интерфейс сервера – REST API с префиксом `/api`. Запросы и ответы передаются в формате JSON, идентификаторы – GUID, статусы – строки (`Created`, `Assigned`, `Accepted`, `InTransit`, `Delivered`, `Failed`, `Cancelled`). Ресурсы сгруппированы по предметным областям; основные конечные точки сведены в таблицу 9, полная спецификация приведена в приложении В.

Таблица 9 – Основные конечные точки API

Метод и путь	Роль	Назначение
`POST /api/auth/login`	все	Вход, получение токена
`GET /api/state`	все	Снимок данных, видимых текущей роли
`GET /api/health`	все	Проверка работоспособности сервера
`POST /api/orders`	client	Создание заказа и автоназначение курьера
`PUT /api/orders/{id}`	client, operator	Изменение состава заказа до принятия курьером
`GET /api/deliveries`	все	Список доставок с фильтрами `q`, `status`, `courierId`, `customerId`
`POST /api/deliveries/{id}/assign`	operator	Назначение курьера (явное или автоматическое)

Метод и путь	Роль	Назначение
PATCH /api/deliveries/{id}/status	все	Смена статуса с проверкой прав и переходов
POST /api/deliveries/{id}/confirm	client, operator	Подтверждение получения заказа
GET/POST/PUT/DELETE /api/customers	operator	Справочник клиентов и их адресов
GET/POST/PUT/DELETE /api/couriers	operator	Справочник курьеров, PATCH …/state – состояние
GET/POST/PUT/DELETE /api/products	operator (чтение – все)	Справочник товаров

Единый формат ошибок – одно из ключевых проектных решений. Любая неудача возвращается как объект { "success": false, "errorCode": "…", "error": "…", "message": "…" }, где errorCode – машинно-читаемый код (около тридцати кодов), а message – готовое сообщение на русском языке, которое клиент показывает пользователю без преобразований. Коды сгруппированы по HTTP-статусам (таблица 10): ошибки аутентификации – 401, нарушения прав – 403, отсутствие сущностей – 404, конфликты состояния – 409, некорректные данные – 400.

Таблица 10 – Соответствие кодов ошибок и HTTP-статусов

HTTP	Коды ошибок	Смысл
401	UNAUTHORIZED, INVALID_CREDENTIALS	Нет входа или неверный пароль
403	FORBIDDEN, UNAUTHORIZED_STATUS_CHANGE	Недостаточно прав на действие
404	ORDER_NOT_FOUND, DELIVERY_NOT_FOUND, COURIER_NOT_FOUND, CUSTOMER_NOT_FOUND и др.	Сущность не найдена
409	COURIER_NOT_AVAILABLE, COURIER_BUSY, INVALID_STATUS_TRANSITION, DELIVERY_ALREADY_*, ORDER_NOT_EDITABLE, LOGIN_TAKEN, *_HAS_ORDERS	Конфликт с текущим состоянием
400	EMPTY_ORDER, INVALID_QUANTITY, INVALID_ADDRESS, INVALID_NAME, FAILURE_REASON_REQUIRED и др.	Некорректные данные запроса

Контракты между модулями внутри сервера описаны в проекте `Contracts`: перечисления статусов, запрос на создание доставки `CreateDeliveryRequest` (идентификатор доставки, идентификатор клиента, адрес) и интерфейс `IDeliveryService`. Такой подход фиксирует границы модулей: модуль заказов знает только интерфейс доставки, а не его реализацию, что позволяет подменять реализацию в тестах и в будущем вынести модуль в отдельный сервис.

2.6 Разработка клиентской части

Клиентское приложение разработано на WPF [6] в проекте `WpfClient`. Выбор настольного клиента обусловлен ролью оператора: ему нужны быстрые таблицы, поиск и привычные для рабочего места элементы управления. Один и тот же исполняемый файл обслуживает все три роли: после входа окно подстраивается под роль, показывая только доступные действия.

Приложение состоит из окна входа `LoginWindow`, главного окна `MainWindow` и трёх диалогов: оформления заказа `OrderDialog`, выбора курьера или клиента из списка `PickDialog` и редактирования данных человека `PersonDialog`. В окне входа пользователь указывает адрес сервера (по умолчанию `http://localhost:8080`), логин и пароль. Главное окно (рисунок 6) содержит таблицу заказов с колонками «№», «Клиент», «Адрес», «Курьер», «Статус», «Сумма» и «Создан», строку поиска, панель действий над выбранным заказом и карточку заказа справа с составом, историей статусов и сведениями о курьере.

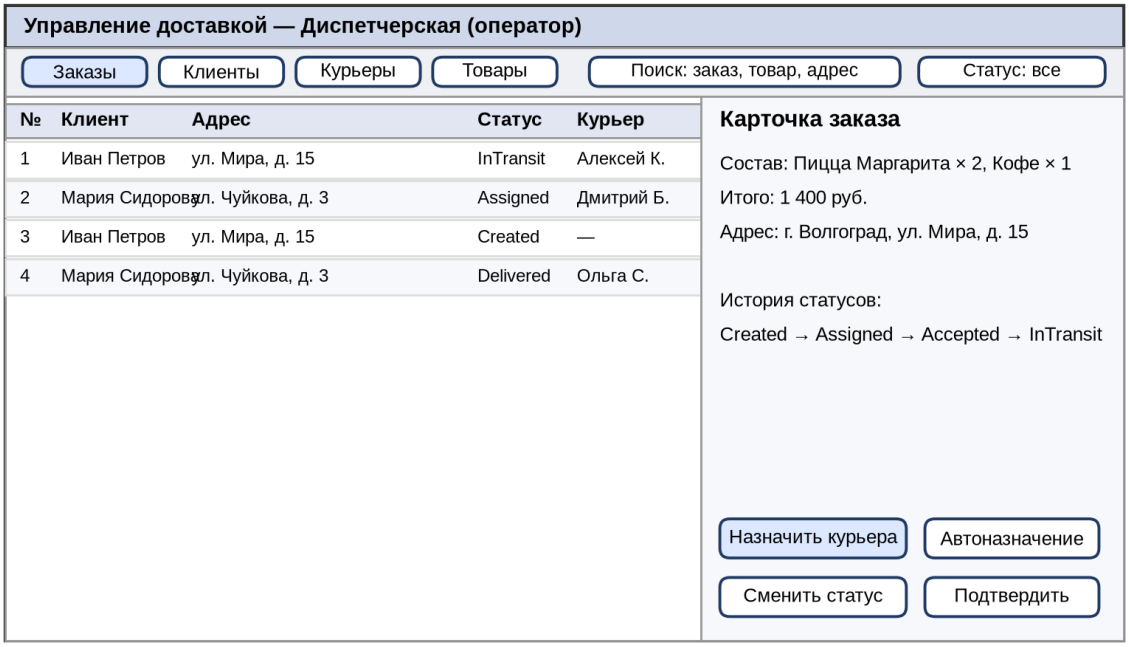


Рисунок 6 – Схема главного окна WPF-клиента

Доступные действия зависят от роли и статуса выбранного заказа. Клиент оформляет заказ, редактирует его до принятия курьером, отменяет и подтверждает получение. Оператор ведёт справочники, назначает курьеров вручную и просматривает все доставки. Курьер видит только свои доставки и последовательно переводит их в статусы «Принята», «В пути», «Доставлена» или «Не выполнена» (с обязательной причиной). Недоступные кнопки отключаются, а если пользователь всё же отправит недопустимый запрос, сервер вернёт ошибку, которая будет показана на русском языке.

Весь сетевой обмен инкапсулирован в классе `Api` (листинг 5). Он создаёт один `HttpClient` с таймаутом 15 секунд, подставляет заголовок `Bearer` после входа и преобразует любой неуспешный ответ в исключение `ApiException` с кодом и сообщением сервера. Сетевые сбои (недоступный сервер, таймаут) превращаются в исключение с кодом `NO_CONNECTION` и понятным текстом «Нет связи с сервером. Проверьте адрес и что ApiHost запущен».

Листинг 5 – Отправка запроса и разбор ошибок в клиенте (src/WpfClient/Api.cs)

```
private async Task<JsonNode?> SendAsync(HttpMethod m, string path, object? body)
{
    using var req = new HttpRequestMessage(m, path);
    if (body is not null)
        req.Content = new StringContent(JsonSerializer.Serialize(body,
            new JsonSerializerOptions(JsonSerializerDefaults.Web)),
            Encoding.UTF8, "application/json");
    HttpResponseMessage resp;
    try { resp = await _http.SendAsync(req); }
    catch (Exception ex) when (ex is HttpRequestException or TaskCanceledException)
    { throw new ApiException("NO_CONNECTION", "Нет связи с сервером. ..."); }
    var text = await resp.Content.ReadAsStringAsync();
```

```
    JsonNode? json = null;
    if (!string.IsNullOrWhiteSpace(text)) { try { json = JsonNode.Parse(text); }
        catch { } }
    if (!resp.IsSuccessStatusCode)
        throw new ApiException(json?["errorCode"]?.ToString() ?? "ERROR",
                               json?["message"]?.ToString() ?? $"Ошибка
                                   {(int)resp.StatusCode}");
    return json;
}
```

Данные в главном окне обновляются автоматически: таймер `DispatcherTimer` с интервалом 8 секунд запрашивает `GET /api/state`, поэтому курьер и оператор видят изменения статусов без ручного обновления. Для запроса используется один эндпоинт, возвращающий снимок данных, отфильтрованный по роли, – это сокращает число обращений к серверу и гарантирует, что клиент не получит чужие данные.

2.7 Интеграция клиентской и серверной частей

Клиент и сервер интегрируются только через REST API и общий формат ошибок, описанные в подразделе 2.5. Сетевой уровень тоньше любого другого уровня приложения, поэтому именно здесь чаще всего возникают сбои конфигурации. Чтобы снизить риск, приняты следующие меры: адрес сервера по умолчанию одинаков во всех местах (клиент, сервер, Docker, скрипты и документация – порт 8080); клиент явно сообщает о недоступности сервера; проверка работоспособности доступна без входа (`GET /api/health`).

Последовательность работы клиента при типовом сценарии «клиент оформляет заказ» следующая: 1) пользователь вводит логин и пароль, клиент отправляет `POST /api/auth/login` и получает токен, роль и идентификатор клиента; 2) клиент запрашивает `GET /api/state` и отображает справочники и свои заказы; 3) в диалоге оформления пользователь выбирает товары и адрес, клиент отправляет `POST /api/orders`; 4) сервер в транзакции сохраняет заказ, создаёт доставку и пытается назначить свободного курьера; 5) клиент показывает сообщение из ответа («Заказ создан, курьер назначен») и обновляет таблицу.

Безопасность обмена обеспечивается на двух уровнях. Сервер проверяет токен и роль при каждом запросе, а также принадлежность данных: клиент может создавать заказы только от своего имени, а курьер – менять статусы только своих доставок. Пароли не передаются повторно после входа, а в базе хранятся в виде соли и хэша PBKDF2. В промышленной эксплуатации обмен следует дополнительно защитить протоколом HTTPS, а сроки жизни токенов и политику паролей – ужесточить; подходы к безопасному обмену данными в клиент-серверных приложениях рассмотрены в [15].

2.8 Обработка ошибок и журналирование

Обработка ошибок построена по принципу «ожидаемые ошибки – коды, неожиданные – исключения». Методы `Store` и модульные правила возвращают код ошибки или `null` вместо исключений для всех ожидаемых нарушений бизнес-правил (пустой заказ, занятый курьер, недопустимый переход), что делает поток управления явным и дешёвым. Обработчик конечной точки преобразует код в HTTP-ответ единой функцией `Err` (листинг 1), поэтому формат и статусы ответов не зависят от места возникновения ошибки.

На стороне клиента ошибки разделены на три группы. Ошибки бизнес-правил приходят от сервера с кодом и сообщением и показываются пользователю в окне сообщения. Сетевые ошибки преобразуются в `NO_CONNECTION`. Все остальные необработанные исключения перехватываются глобальным обработчиком `DispatcherUnhandledException` в классе `App`, который показывает сообщение вместо аварийного завершения программы и помечает исключение обработанным.

Журналирование выполняется через стандартный интерфейс `ILogger` платформы .NET. Сервер пишет в консоль ключевые события жизненного цикла заказа: создание заказа и доставки, смену статусов с указанием предыдущего и нового значения, назначение курьера. В модуле доставки используются области журнала (`BeginScope`) с идентификаторами доставки и клиента, поэтому все сообщения одной операции можно связать между собой. Уровни сообщений соответствуют серьёзности: `Information` – нормальные события, `Warning` – отклонённые запросы (повторное создание, некорректные данные). В контейнере вывод журнала доступен командой `docker compose logs`.

Дополнительным механизмом наблюдаемости служит история статусов. Таблицы `status_history` и `order_history` хранят каждое изменение с временем и комментарием, поэтому по базе данных можно восстановить, кто и когда перевёл доставку в тот или иной статус, и разобрать причину неудачной доставки.

2.9 Конфигурирование

Конфигурация сервера задаётся стандартными средствами ASP.NET Core: переменные окружения, аргументы командной строки и файлы `appsettings`. Приложению нужны всего два параметра: адрес прослушивания и строка подключения к базе данных (таблица 11). Остальные значения (срок сессии, число итераций PBKDF2, максимальное количество товара в позиции) являются константами предметной области и вынесены в код модулей.

Таблица 11 – Параметры конфигурации

Параметр	Значение по умолчанию	Назначение
ASPNETCORE_URLS	http://localhost:8080; в Docker – http://+:8080	Адрес и порт прослушивания сервера
ConnectionStrings__Default	Host=db;Port=5432;Database=delivery;Username=delivery;Password=… (в Docker); для локального запуска – localhost	Строка подключения к PostgreSQL (обязательна)
Адрес сервера в клиенте	http://localhost:8080	Поле в окне входа WPF-клиента
ASPNETCORE_ENVIRONMENT	Production в образе; Development в docker-compose.yml	Режим выполнения ASP.NET Core
Seed__Demo	true в разработке, false в Production	Демонстрационные пользователи и данные
POSTGRES_PASSWORD, OPERATOR_PASSWORD	обязательны в docker-compose.prod.yml	Пароль базы и первого оператора (не короче 12 символов)
ALLOW_REGISTRATION, MAX_REGISTRATIONS_PER_HOUR	true, 20	Самостоятельная регистрация клиентов и её лимит
TRUSTED_PROXIES, TRUSTED_NETWORKS	пусто	Доверенные прокси, от которых принимается X-Forwarded-For

Двойное подчёркивание в имени ConnectionStrings__Default – стандартное правило .NET для вложенных ключей в переменных окружения, оно соответствует ключу ConnectionStrings:Default в конфигурации. Секретные значения (пароли, ключи) в репозитории не хранятся; демонстрационные учётные записи создаются только при пустой базе данных и предназначены для учебного запуска.

Запуск без Docker требует работающего PostgreSQL (его можно поднять командой docker compose up db) и выполняется скриптами run.bat и run.sh (сервер) и run-client.bat (клиент). Из Visual Studio можно запустить проект ApiHost (в этом случае адрес берётся из launchSettings.json, где также указан порт 8080) и затем WpfClient. Для запуска обоих проектов одновременно в Visual Studio настраиваются несколько запускаемых проектов.

3 ТЕСТИРОВАНИЕ, ОТЛАДКА И РАЗВЁРТЫВАНИЕ

3.1 Стратегия тестирования

Качество системы обеспечивается автоматическими тестами, которые запускаются одной командой `dotnet test` и выполняются за несколько секунд. Такой подход соответствует практикам непрерывной интеграции: любое изменение можно быстро проверить на регрессии, а автоматизация тестирования сокращает время обратной связи и стоимость исправления дефектов [14]. Для тестов использован фреймворк xUnit [9].

Стратегия строится по принципу пирамиды тестов (таблица 12). В основании находятся быстрые модульные тесты чистых правил предметной области – они не используют ни сеть, ни базу данных. Выше расположены интеграционные тесты, которые проверяют работу хранилища, транзакций и бизнес-операций целиком на реальной базе PostgreSQL. Ручное тестирование интерфейса WPF выполняется по сценариям подраздела 1.4 и дополняет автоматические проверки.

Таблица 12 – Уровни тестирования

Уровень	Проект	Что проверяется	Тестов
Модульный	`tests/ OrderModule.Tests`	Правила заказа, доставки, курьера и клиента; расчёт денег в копейках	41
Интеграционный	`tests/ Integration.Tests`	Store и HTTP API с реальной PostgreSQL: сценарии ТЗ, роли, валидация, идемпотентность, миграции, безопасность, регистрация, фоновое назначение, настройки запуска	236
Ручной	WPF-клиент и сервер	Сценарии использования, сообщения об ошибках, обновление таблицы	по сценариям
Итого автоматических			**277**

Тестовые данные изолированы: каждый интеграционный тест создаёт собственную базу PostgreSQL со случайным именем и удаляет её после завершения, поэтому тесты не зависят друг от друга, могут выполняться в любом порядке и не затрагивают рабочую базу. Использование реальной базы вместо подставной (mock) повышает достоверность: проверяется настоящий SQL, ограничения схемы, миграции и поведение транзакций, а не предположения о них.

Критерии качества тестирования: все 277 автоматических тестов проходят, каждое бизнес-правило из требований подраздела 1.3 проверено хотя бы одним тестом (соответствие требований и тестов ведётся в матрице docs/tz-traceability.md), для каждой ошибочной ситуации проверяется не только отказ, но и конкретный код ошибки.

3.2 Модульное тестирование

Модульные тесты расположены в проекте OrderModule.Tests и разделены на два класса: RulesTests проверяет статические правила модулей, MoneyTests – точность денежных расчётов в копейках. Параметризованные тесты ([Theory]) позволяют описать множество случаев таблицей данных, поэтому 41 тест покрывает значительно больше ситуаций, чем количество методов (таблица 13).

Таблица 13 – Модульные тесты

Тест	Случаев	Проверяемое поведение
`DeliveryTransitions`	8	Разрешённые и запрещённые переходы статусов доставки
`OrderStatusFollowsDelivery`	7	Соответствие статуса заказа статусу доставки
`FailedDelivery_IsNotCancelledOrder`	1	Неудачная доставка не превращается в отменённый заказ
`EveryDeliveryStatus_HasOrderStatus_…`	1	У каждого статуса есть соответствие, финальные статусы без переходов
`FinalStatuses, OrderQuantities`	2	Финальные статусы; количество позиций 1–99
`ClientCancelAndEditWindow`	1	Окно редактирования и отмены заказа клиентом
`CourierStateRules, CustomerRulesWork`	2	Состояния курьера; проверка имени и адреса клиента
`ToKopecks_IsExact, ToKopecks_RoundsHalfAwayFromZero`	11	Перевод рублей в копейки без погрешности, округление
`IsValidPrice`	4	Допустимые значения цены
`RoundTrip…, Total_…`	4	Сумма заказа в копейках, переполнение

Пример параметризованного теста приведён в листинге 6. Данные теста записаны тройками «исходный статус – целевой статус – ожидаемый результат», что делает тест одновременно и спецификацией диаграммы состояний (рисунок 3).

Листинг 6 – Параметризованный тест переходов статусов (tests/OrderModule.Tests/RulesTests.cs)

```
[Theory]
[InlineData(DeliveryStatus.Created,   DeliveryStatus.Assigned,  true)]
[InlineData(DeliveryStatus.Assigned,  DeliveryStatus.Accepted,  true)]
[InlineData(DeliveryStatus.Accepted,  DeliveryStatus.InTransit, true)]
[InlineData(DeliveryStatus.InTransit, DeliveryStatus.Delivered, true)]
[InlineData(DeliveryStatus.InTransit, DeliveryStatus.Failed,    true)]
[InlineData(DeliveryStatus.Created,   DeliveryStatus.Delivered, false)]
[InlineData(DeliveryStatus.Accepted,  DeliveryStatus.Cancelled, false)]
[InlineData(DeliveryStatus.Delivered, DeliveryStatus.Created,   false)]
public void DeliveryTransitions(DeliveryStatus from, DeliveryStatus to, bool ok) =>
    Assert.Equal(ok, DeliveryRules.CanTransition(from, to));
```

3.3 Интеграционное тестирование

Интеграционные тесты проверяют систему так, как её использует сервер: через публичные методы Store и по HTTP через API с реальной базой данных PostgreSQL. Класс ScenarioTests прямо воспроизводит сценарии из технического задания: сценарий 1 – создание заказа с автоматическим назначением свободного курьера, сценарий 2 – создание заказа при отсутствии свободных курьеров с последующим повторным назначением, сценарий 3 – полный жизненный цикл от создания до доставки с проверкой истории статусов и временных меток. Остальные классы проверяют роли и права, проверку входных данных, идемпотентность и параллельные запросы, миграции, безопасность учётных записей, регистрацию, фоновое назначение курьеров и настройки запуска (таблица 14).

Таблица 14 – Группы интеграционных тестов

Группа	Тесты	Проверка
Сценарии ТЗ	`ScenarioTests (22)`	Автоназначение, создание без курьеров и повторное назначение, полный цикл, права, статусы, справочники, поиск
HTTP API и роли	`ApiRoleTests, ApiValidationTests, ApiScenarioTests, ValidationTests (72)`	Коды 401/403, формат ошибок, проверка JSON, enum, длин полей и поиска, жизненный цикл по HTTP
Идемпотентность и надёжность	`IdempotencyTests, IdempotencyApiTests, ReliabilityTests (29)`	Повтор requestId, 409 при других данных, откат при сбое, параллельные запросы
Фоновое назначение	`AssignmentRecoveryTests, AssignmentRetryServiceTests, AssignmentAndRetryTests и др. (38)`	Очередь ожидающих доставок, нарастающая пауза, сбой БД, гонки назначений

Группа	Тесты	Проверка
Безопасность и регистрация	`AccountSecurityTests`, `SessionAndPasswordTests`, `RegistrationTests` (32)	Блокировка входа, сессии в базе, смена пароля, регистрация клиентов, доверенный прокси
Миграции и запуск	`MigrationTests`, `StartupConfigTests`, `ProductionSeedTests` (35)	Применение и откат миграций, отказ слабых настроек в Production, демо-данные
Журнал изменений	`ChangeLogTests` (8)	Запись старых и новых значений заказа, автор и время

Полный прогон дал результат: 41 модульный тест за 0,1 с и 236 интеграционных тестов за 77 с, все пройдены, предупреждений компилятора нет. Полный перечень тестов по классам приведён в приложении Д. Клиент WPF на Linux собирается с параметром EnableWindowsTargeting, а запускается только в Windows.

3.4 Отладка программной системы

Отладка выполнялась по мере разработки: ошибки обнаруживались тестами, сообщениями компилятора, при ручном запуске системы и при инспекции кода. Для локализации дефектов использовались точки останова в Visual Studio, журнал сервера, просмотр ответов API (код и тело ответа), а также прямое чтение базы данных PostgreSQL. Практика показала, что большинство проблем возникло не в бизнес-логике, а на границах: конфигурация запуска, схема данных, состав ресурсов интерфейса.

Выявленные дефекты и принятые меры сведены в таблицу 15. Наиболее показателен дефект № 6: после переписывания клиента на WPF клиент по умолчанию обращался к порту 8080, а запуск проекта `ApiHost` из Visual Studio использовал порт 5000 – пользователь получал сообщение «Нет связи с сервером» при полностью исправной системе. Причина найдена сопоставлением адреса в окне входа и адреса, который выводит сервер при старте. Исправление системное, а не разовое: порт по умолчанию сервера теперь 8080, и значение совпадает во всех файлах конфигурации.

Таблица 15 – Выявленные дефекты и способы устранения

№	Дефект	Проявление	Устранение
1	Запуск Visual Studio не с того стартового проекта	Запускалась библиотека классов вместо сервера	Указан стартовый проект `ApiHost`, порядок запуска описан в README
2	Переименование статуса `PickedUp`	Ошибки компиляции и несоответствие документации	Статус заменён на `InTransit` везде, включая тесты и документацию
3	Старая схема базы данных (версия 1)	Ошибки SQL при запуске на файле от прежней версии	Версия схемы в `PRAGMA user_version`; при несовпадении схема пересоздаётся
4	Остатки шаблона (`WeatherForecast`, `Class1`)	Лишний код и лишние маршруты в API	Шаблонные файлы удалены
5	Дубликат ресурса WPF `Muted`	Исключение при загрузке окна	Ресурс переименован в `MutedText`
6	Несовпадение портов клиента (8080) и сервера из Visual Studio (5000)	«Нет связи с сервером» при работающем сервере	Порт по умолчанию 8080 в `Program.cs` и `launchSettings.json`
7	Предупреждение о `WebRootPath` (остатки веб-клиента)	Предупреждение при старте сервера	Удалены `UseDefaultFiles` и `UseStaticFiles`, каталог `wwwroot` не нужен
8	Расхождение окончаний строк с `.editorconfig`	Замечания форматирования (LF вместо CRLF)	Выполнен `dotnet format whitespace`, проверка проходит
9	Миграция 0006 падала, если у курьера уже было несколько активных доставок	Сервер не запускался на таких данных	Лишние доставки возвращаются в очередь до создания уникального индекса
10	Счётчики попыток входа и регистрации хранились в памяти процесса	Лимит сбрасывался при перезапуске и не работал при нескольких экземплярах	Счётчики в таблице rate_limits, ключ «логин + адрес»
11	Числовые значения статусов принимались как enum	Неверный статус мог быть принят без ошибки	JsonStringEnumConverter без чисел, проверка Enum.IsDefined, код 400
12	Не было ограничений длины полей; % и _ в поиске работали как шаблоны	Слишком длинные значения в базе, неточный поиск	Ограничения длины с отдельными кодами ошибок, экранирование LIKE

№	Дефект	Проявление	Устранение
13	X-Forwarded-For принимался от любого клиента	Адрес для лимитов можно было подделать	Заголовок учитывается только от доверенных прокси
14	WPF терял адрес при изменении заказа и не принимал адрес сервера без http://	Заказ менялся без адреса, «Нет связи с сервером»	Подставляется текущий адрес, http:// добавляется автоматически

После исправления проверена связка «клиентский адрес – сервер»: сервер, запущенный без дополнительных параметров, слушает `http://localhost:8080`, а вход под учётной записью `ivan` возвращает токен и роль `client` (подраздел 3.8).

3.5 Инспекция и повышение качества программного кода

Инспекция кода выполнена по чек-листу этапа 5 курса: проверялись именование, ответственность классов, направление зависимостей, дублирование, обработка `null`, валидация, обработка исключений, конфигурация и секреты, наличие тестов, мёртвый код и соответствие `.editorconfig`. Результаты зафиксированы в документе `docs/stage-5-report.md` в виде таблицы из 12 замечаний. Их текущее состояние показано в таблице 16.

Таблица 16 – Состояние замечаний инспекции

ID	Замечание	Состояние
INS-01	Незащищённый `Dictionary` в `DeliveryService` при использовании как singleton	Учтено: рабочий путь использует `Store` с блокировкой и транзакциями; сервис остаётся учебной реализацией контракта
INS-02, INS-03, INS-07, INS-11	Нет `CancellationToken`, неполный DTO, результат `bool` без причины	Оставлено как направление развития (см. заключение); на работу системы не влияет
INS-04, INS-05	Параметры `POST /api/orders` без DTO, единый код отказа	Исправлено: DTO `CreateOrderRequest`, около тридцати кодов ошибок
INS-06	Шаблон `WeatherForecast` в API	Исправлено: файлы удалены
INS-08	Недостаточно тестов	Исправлено: 277 тестов вместо базового набора
INS-09	Форматирование не соответствует `.editorconfig`	Исправлено: `dotnet format whitespace`

ID	Замечание	Состояние
INS-10	Версия `Microsoft.Extensions.Logging` 10.0.12 при `net9.0`	Оставлено: сборка и тесты проходят без предупреждений; рекомендуется выровнять версию до 9.x
INS-12	README пуст	Исправлено: README описывает запуск, API, тесты, Docker

Помимо исправления замечаний повышению качества служат следующие практики: статические правила предметной области, не зависящие от инфраструктуры; единый формат ошибок; параметризованные SQL-запросы; транзакции для составных операций; автоматическая проверка форматирования; описание зависимостей и версий в `docs/dependencies.md`; записи архитектурных решений. Сборка в конфигурации Release выполняется без предупреждений и ошибок.

3.6 Контейнеризация программной системы

Контейнеризация упаковывает приложение вместе с окружением в образ, который одинаково запускается на любой машине с Docker [8]. Это устраняет типичную проблему «у меня работает» и упрощает развёртывание, тестирование и откат версий [12, 16]. В проекте контейнеризуется только серверная часть: клиент WPF – настольное приложение для Windows и запускается на рабочих местах пользователей.

Образ собирается по двухэтапной схеме (полный текст `Dockerfile` – в приложении Г). На первом этапе используется образ `mcr.microsoft.com/dotnet/sdk:9.0`: в него копируется исходный код и командой `dotnet publish` собирается Release-версия `ApiHost`. На втором этапе берётся лёгкий образ `mcr.microsoft.com/dotnet/aspnet:9.0`, содержащий только среду выполнения, и в него копируется результат публикации. Благодаря этому итоговый образ не содержит SDK, исходников и промежуточных файлов, что уменьшает его размер и поверхность атаки.

Образ не содержит данных и настроек базы: строка подключения передаётся переменной ConnectionStrings__Default. Порт 8080 объявлен публикуемым (EXPOSE 8080), по умолчанию задан режим Production (без строки подключения и с демонстрационными данными сервер не стартует), процесс запускается не от root (пользователь app из базового образа). Точка входа – dotnet ApiHost.dll.

Оркестровка описана файлом docker-compose.yml (приложение Г). В нём два сервиса: db (postgres:16 с проверкой готовности и именованным томом pgdata) и

app, который запускается после готовности базы и публикуется только на 127.0.0.1. Для сервера предназначен отдельный файл docker-compose.prod.yml: пароли задаются через .env и обязательны, база снаружи недоступна, демонстрационные данные отключены, адрес клиента из X-Forwarded-For принимается только от доверенных прокси. Для масштабирования на несколько узлов можно использовать оркестраторы – Docker Swarm или Kubernetes; их сравнение приведено в [17], а благодаря хранению сессий и счётчиков в PostgreSQL несколько экземпляров API работают с общей базой без дополнительных настроек.

3.7 Развёртывание системы с использованием Docker

Для развёртывания на сервере необходим установленный Docker с подсистемой Compose. Порядок действий состоит из четырёх шагов (листинг 7): получение исходного кода, сборка образа и запуск контейнера в фоновом режиме, проверка состояния и просмотр журнала.

Листинг 7 – Команды развёртывания

```
git clone https://github.com/Levikaci/order-delivery-managment-system.git
cd order-delivery-managment-system


docker compose up --build -d      # сборка образа и запуск контейнера
docker compose ps                 # состояние сервиса app
curl http://localhost:8080/api/health
docker compose logs -f app        # журнал сервера


docker compose down               # остановка (данные сохраняются в томе)
docker compose down -v            # остановка с удалением данных
```

Схема развёртывания показана на рисунке 7. Клиентские приложения WPF на рабочих местах обращаются по HTTP к порту 8080 хоста Docker; запрос попадает в контейнер app, где его обслуживает ASP.NET Core, а данные сохраняются в PostgreSQL в контейнере db на томе pgdata. Для сети предприятия адрес сервера вводится в окне входа клиента (например, 192.168.0.10:8080: префикс http:// клиент добавляет сам).

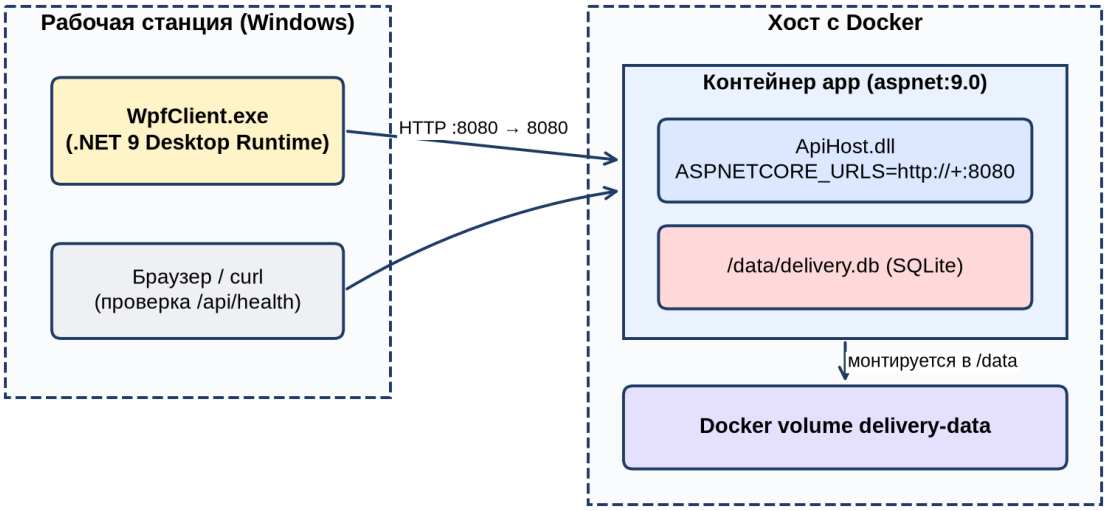


Рисунок 7 – Схема развёртывания системы

Обновление версии выполняется повторной командой docker compose up --build -d: образ пересобирается, контейнер приложения пересоздаётся, а данные остаются в томе pgdata. Недостающие миграции применяются при запуске. Резервное копирование выполняется стандартными средствами PostgreSQL (pg_dump). Без Docker сервер запускается скриптами run.bat и run.sh либо командой dotnet run --project src/ApiHost при доступном PostgreSQL.

3.8 Проверка работоспособности развёрнутой системы

Работоспособность проверена на опубликованной сборке сервера (`dotnet publish -c Release`), запущенной с теми же переменными окружения, что заданы в `Dockerfile`: `ASPNETCORE_URLS=http://+:8080` и отдельный файл базы данных. Сборка и запуск образа командой `docker compose up --build` в среде подготовки этой записки не выполнялись, так как Docker в ней недоступен; команда приведена в подразделе 3.7 и должна быть выполнена при защите на целевой машине. Проверялись конечные точки, которые составляют основной сценарий и ключевые защитные механизмы (таблица 17).

Таблица 17 – Проверка работоспособности сервера

№	Проверка	Запрос	Ожидаемый и полученный результат
1	Доступность сервера	`GET /api/health`	200, `{"status":"ok"}` – получено
2	Вход клиента	`POST /api/auth/login` (`ivan/ivan`)	200, токен, роль `client` – получено

№	Проверка	Запрос	Ожидаемый и полученный результат
3	Создание заказа	`POST /api/orders` с токеном клиента	200, статус `Assigned`, заказ `Confirmed`, `courierAssigned=true` – получено
4	Заказ без позиций	`POST /api/orders`, пустой список	400, `EMPTY_ORDER` – получено
5	Доступ без входа	`GET /api/state` без токена	401, `UNAUTHORIZED` – получено
6	Автотесты	`dotnet test`	277 из 277 пройдены – получено

Все шесть проверок дали ожидаемый результат. Автоматическое назначение курьера, смена статуса заказа вместе со статусом доставки, единый формат ошибок и защита конечных точек от анонимного доступа подтверждены на работающем сервере. Проверка WPF-клиента выполняется вручную в Windows по сценариям подраздела 1.4: вход, оформление заказа, назначение и прохождение статусов курьером, подтверждение получения.

Нагрузочное тестирование не выполнялось; проверена корректность при параллельных запросах (одновременное назначение курьера, повтор запросов с одним requestId, несколько фоновых обработчиков).

ЗАКЛЮЧЕНИЕ

В ходе курсового проекта разработана клиент-серверная программная система управления доставкой заказов, которая автоматизирует приём заказов, назначение курьеров, контроль статусов доставки и подтверждение получения. Поставленная цель достигнута, все задачи, сформулированные во введении, решены.

Выполнен анализ предметной области, определены границы системы, сформулированы функциональные и нефункциональные требования, описаны сценарии использования для трёх ролей – клиента, оператора и курьера. Спроектирована модульная архитектура, в которой предметные области (заказы, доставка, курьеры, клиенты) вынесены в отдельные библиотеки с контрактами взаимодействия, спроектирована схема базы данных из таблиц справочников, заказов, доставок и историй статусов, обоснован выбор технологий.

Разработаны серверная часть на ASP.NET Core 9 с хранилищем PostgreSQL и REST API (около сорока конечных точек, единый формат ошибок, аутентификация по токену с сессиями в базе, разграничение доступа по ролям, самостоятельная регистрация клиентов) и клиентское приложение на WPF. Бизнес-операции выполняются в транзакциях, создание заказа идемпотентно, назначение курьеров повторяется в фоне, пароли хранятся в виде хэша PBKDF2, обработка ошибок и журналирование организованы единообразно. Общий объём исходного кода составляет около 3 200 строк на C#.

Качество проверено 277 автоматическими тестами (41 модульный и 236 интеграционных на реальной базе PostgreSQL), все они проходят. Проведены отладка и инспекция кода: выявлены и устранены четырнадцать дефектов, включая несовпадение портов клиента и сервера, падение миграции на данных с лишними активными доставками и хранение счётчиков входа в памяти процесса. Подготовлена контейнеризация: двухэтапный Dockerfile, docker-compose.yml для разработки и docker-compose.prod.yml для рабочего развёртывания.

Практическая значимость работы состоит в том, что получен законченный проект с открытым исходным кодом, документацией, тестами и воспроизводимым развёртыванием, пригодный для демонстрации и дальнейшего развития. Дальнейшее развитие возможно по следующим направлениям:

– добавить уведомления клиентам об изменении статуса заказа вместо опроса сервера раз в несколько секунд и перенести секреты в специальное хранилище;

– добавить защищённый обмен по HTTPS, ограничение числа попыток входа и ротацию токенов;

– добавить в контракты `CancellationToken` и типизированные результаты операций вместо значения `bool`;

– реализовать уведомления клиента о смене статуса и отображение маршрута на карте;

– настроить непрерывную интеграцию, которая при каждом коммите собирает решение, запускает тесты и собирает Docker-образ.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 2.105-2019. Единая система конструкторской документации. Общие требования к текстовым документам. – Введ. 2020-03-01. – М.: Стандартинформ, 2019. – 32 с.

2. ГОСТ 2.104-2006. Единая система конструкторской документации. Основные надписи. – Введ. 2006-09-01. – М.: Стандартинформ, 2007. – 11 с.

3. ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. – Введ. 2021-01-01. – М.: Стандартинформ, 2020. – 20 с.

4. Троелсен, Э. Язык программирования C# 9 и платформа .NET 5: основные принципы и практики программирования: пер. с англ. / Э. Троелсен, Ф. Джепикс. – 10-е изд. – М.: Диалектика (Вильямс), 2022. – 1392 с.

5. Документация по ASP.NET Core [Электронный ресурс] // Microsoft Learn. – Режим доступа: https://learn.microsoft.com/aspnet/core, свободный (дата обращения: 09.11.2026).

6. Документация по Windows Presentation Foundation [Электронный ресурс] // Microsoft Learn. – Режим доступа: https://learn.microsoft.com/dotnet/desktop/wpf/, свободный (дата обращения: 09.11.2026).

7. PostgreSQL 16 Documentation [Электронный ресурс]. – Режим доступа: https://www.postgresql.org/docs/16/, свободный (дата обращения: 12.11.2026).

8. Docker Docs [Электронный ресурс]. – Режим доступа: https://docs.docker.com/, свободный (дата обращения: 09.11.2026).

9. xUnit.net Documentation [Электронный ресурс]. – Режим доступа: https://xunit.net/, свободный (дата обращения: 09.11.2026).

10. Git Documentation [Электронный ресурс]. – Режим доступа: https://git-scm.com/doc, свободный (дата обращения: 09.11.2026).

11. Аркабаев, Н.К. Разработка WEB серверных приложений на базе .NET Core в примере интернет-магазина / Н.К. Аркабаев, З.Ж. Алымова // Вестник Ошского государственного университета. – 2024. – № 1. – С. 142–154. – DOI 10.52754/16948610_2024_1_13.

12. Лю, Ю. Контейнеризация и автоматизация развертывания веб-приложений: анализ и практическая реализация / Ю. Лю, В.И. Борисов // Computational Nanotechnology. – 2025. – Т. 12, № 1. – С. 194–201. – DOI 10.33693/2313-223X-2025-12-1-194-201. – EDN NFNIFX.

13. Мусатов, А.О. Архитектурная трансформация при построении масштабируемых CRM-систем от стартапа до enterprise / А.О. Мусатов // Universum: технические науки. – 2025. – № 9(138). – DOI 10.32743/UniTech.2025.138.9.20720.

14. Тюменцев, Д. Автоматизация тестирования в DevOps: подходы и лучшие практики / Д. Тюменцев // Международный журнал гуманитарных и естественных наук. – 2024. – № 2-2. – С. 156–159. – DOI 10.24412/2500-1000-2024-2-2-156-159.

15. Хорев, П.Б. Безопасный обмен файлами на основе сетей доверия и сертификатов открытых ключей с помощью разработанного приложения / П.Б. Хорев, Д.А. Лосев // Программные продукты и системы. – 2024. – Т. 37, № 2. – С. 230–237. – DOI 10.15827/0236-235X.142.230-237.

16. Bobunov, A. Using containerization to simplify and accelerate testing processes in financial organizations / A. Bobunov // Международный журнал гуманитарных и естественных наук. – 2024. – № 8-1(95). – С. 113–117. – DOI 10.24412/2500-1000-2024-8-1-113-117.

17. Tiumentsev, D. A Comparative Study of Resource Management Approaches in Kubernetes and Docker Swarm: Efficiency and Scalability / D. Tiumentsev // Бюллетень науки и практики. – 2024. – DOI 10.33619/2414-2948/108/18.

Приложение А

(справочное)

Архитектурная схема программной системы

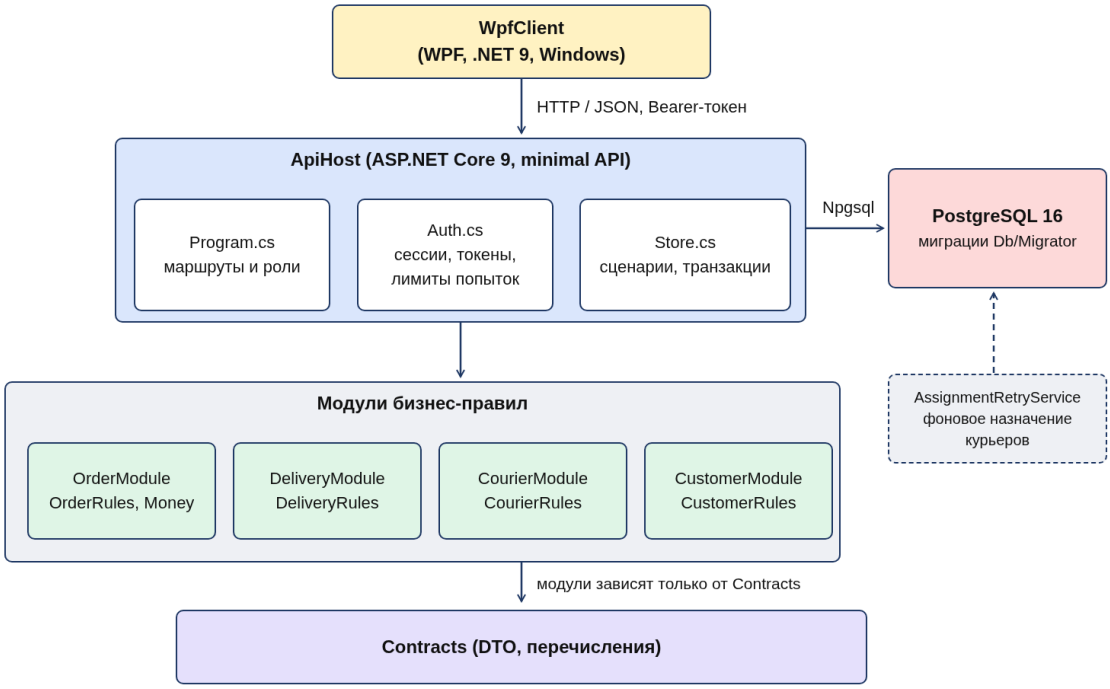


Рисунок А.1 – Архитектурная схема: клиент WPF, сервер ApiHost, модули и база данных PostgreSQL

На схеме показаны три уровня системы: клиентское приложение WPF, серверный проект ApiHost с обработчиками REST API, службой аутентификации и хранилищем, а также библиотеки модулей и контрактов. Данные хранятся в PostgreSQL; фоновая служба назначения курьеров работает в том же процессе сервера. Описание компонентов и их взаимодействия приведено в подразделах 1.5 и 1.6.

Приложение Б

(справочное)

Схема базы данных

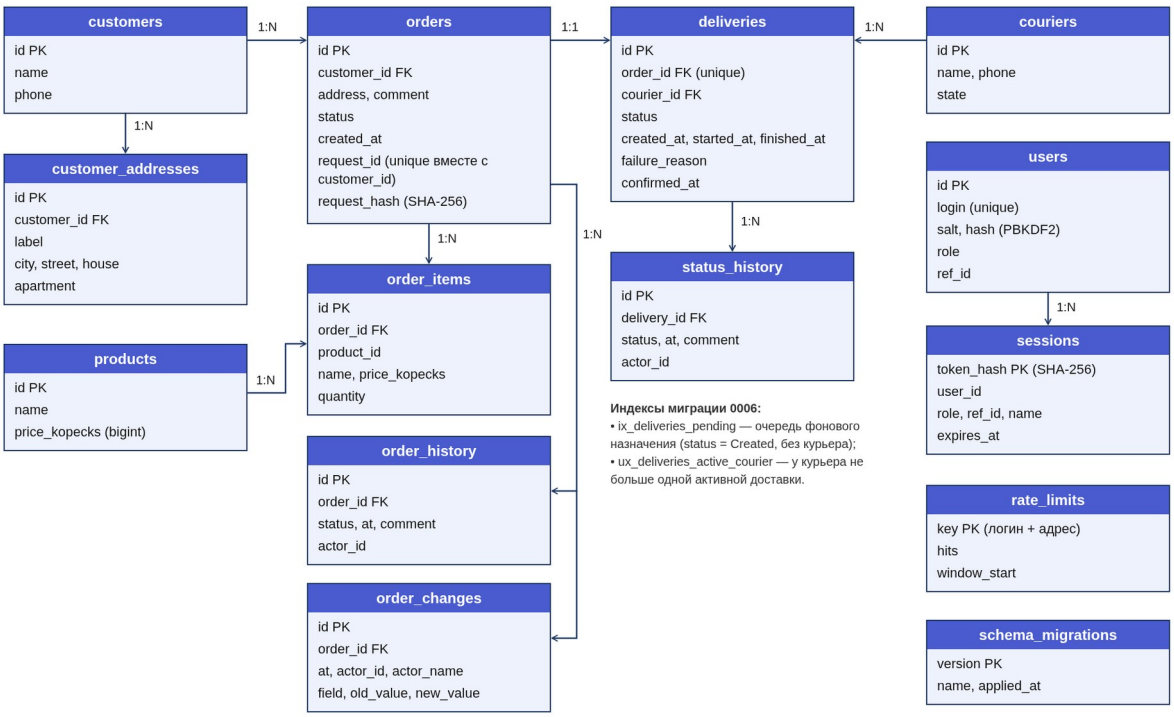


Рисунок Б.1 – Схема базы данных (PostgreSQL, миграции 0001–0007)

База данных содержит таблицы пользователей, клиентов, адресов, курьеров, товаров, заказов и их позиций, доставок, а также таблицы `order_history` и `status_history` для истории изменений. Описание таблиц приведено в таблице 6 подраздела 1.7.

Приложение В

(справочное)

Спецификация API

Все пути указаны относительно адреса сервера (по умолчанию `http://localhost:8080`). Идентификаторы – GUID. Кроме входа и проверки работоспособности, все запросы требуют заголовка `Authorization: Bearer <токен>`. Ошибки возвращаются в формате `{ "success": false, "errorCode": "…", "error": "…", "message": "…" }`.

Таблица В.1 – Конечные точки REST API

Метод	Путь	Роль	Тело запроса	Ответ и ошибки
POST	/api/auth/login	все	`{login, password}`	200: `{token, role, refId, name}`; 401 `INVALID_CREDENTIALS`
POST	/api/auth/logout	вошедший	–	200
GET	/api/auth/me	вошедший	–	200: роль, идентификатор, имя
GET	/api/health	все	–	200: `{status:"ok"}`
GET	/api/state	вошедший	–	200: снимок данных по роли
GET	/api/customers	operator	–	200: список клиентов
POST	/api/customers	operator	`{name, phone, login, password}`	200; 400 `INVALID_NAME`; 409 `LOGIN_TAKEN`
GET	/api/customers/{id}	operator, владелец	–	200; 404 `CUSTOMER_NOT_FOUND`
PUT	/api/customers/{id}	operator	`{name, phone}`	200; 404; 400
DELETE	/api/customers/{id}	operator	–	200; 409 `CUSTOMER_HAS_ORDERS`
GET	/api/customers/{id}/addresses	operator, владелец	–	200: адреса клиента
POST	/api/customers/{id}/addresses	operator, владелец	`{label, city, street, house, apartment}`	200; 400 `INVALID_ADDRESS`
GET	/api/addresses/{id}	operator, владелец	–	200; 404 `ADDRESS_NOT_FOUND`
PUT	/api/addresses/{id}	operator, владелец	как при создании	200; 404; 400
DELETE	/api/addresses/{id}	operator, владелец	–	200; 404
GET	/api/couriers	operator	–	200: список курьеров
GET	/api/couriers/available	operator	–	200: курьеры в состоянии `Available`
POST	/api/couriers	operator	`{name, phone, login, password}`	200; 400; 409
PUT	/api/couriers/{id}	operator	`{name, phone}`	200; 404 `COURIER_NOT_FOUND`
DELETE	/api/couriers/{id}	operator	–	200; 409 `COURIER_HAS_ORDERS`

Метод	Путь	Роль	Тело запроса	Ответ и ошибки
PATCH	/api/couriers/{id}/state	operator, courier	{state}	200; 409 COURIER_BUSY; 400 INVALID_COURIER_STATE; 403
GET	/api/couriers/{id}/ deliveries	operator, courier	–	200: доставки курьера
GET	/api/products	вошедший	–	200: список товаров
POST	/api/products	operator	{name, price}	200; 400 INVALID_PRODUCT
PUT	/api/products/{id}	operator	{name, price}	200; 404 PRODUCT_NOT_FOUND
DELETE	/api/products/{id}	operator	–	200; 404
POST	/api/orders	client	{customerId, items: [{productId, quantity}], deliveryAddress | addressId, comment, saveAddress}	200: {orderId, deliveryId, status, courierAssigned, message}; 400 EMPTY_ORDER, INVALID_QUANTITY, INVALID_ADDRESS
GET	/api/orders/{id}	operator, владелец, курьер заказа	–	200; 404 ORDER_NOT_FOUND; 403
PUT	/api/orders/{id}	operator, client	{items, deliveryAddress, addressId, comment}	200; 409 ORDER_NOT_EDITABLE
POST	/api/deliveries	operator	{orderId}	200; 409 DELIVERY_ALREADY_EXISTS; 404
GET	/api/deliveries	вошедший	параметры q, status, courierId, customerId	200: список (клиент и курьер видят только свои)
GET	/api/deliveries/{id}	operator, владелец, курьер	–	200; 404 DELIVERY_NOT_FOUND
POST	/api/deliveries/{id}/assign	operator	{courierId} (необязательно)	200; 409 COURIER_NOT_AVAILABLE, DELIVERY_ALREADY_ASSIGNED
PATCH	/api/deliveries/{id}/status	вошедший	{status, comment}	200: {previousStatus, status}; 403 UNAUTHORIZED_STATUS_CHANGE; 409 INVALID_STATUS_TRANSITION; 400 FAILURE_REASON_REQUIRED
POST	/api/deliveries/{id}/ confirm	client, operator	–	200; 409 INVALID_STATUS_TRANSITION (до Delivered)

Приложение Г

(обязательное)

Dockerfile и docker-compose.yml

Листинг Г.1 – Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish src/ApiHost/ApiHost.csproj -c Release -o /app
FROM mcr.microsoft.com/dotnet/aspnet:9.0
WORKDIR /app
COPY --from=build /app .
# порт 8080 задаёт базовый образ (ASPNETCORE_HTTP_PORTS); по умолчанию — рабочее
окружение:
# без строки подключения и с демо-данными сервер не стартует
ENV ASPNETCORE_ENVIRONMENT=Production
EXPOSE 8080
# не от root: пользователь app есть в базовом образе ASP.NET
USER $APP_UID
ENTRYPOINT ["dotnet","ApiHost.dll"]
```

Листинг Г.2 – docker-compose.yml

```
# Локальная разработка и учебный запуск: демо-данные, стандартный пароль БД, порты
только на 127.0.0.1.
# Для сервера используйте docker-compose.prod.yml (см. README, раздел «Рабочее
развёртывание»).
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: delivery
      POSTGRES_USER: delivery
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-delivery}
    ports:
      # для запуска API и тестов вне Docker (dotnet run / dotnet test); снаружи
машины база недоступна
      - "127.0.0.1:${DB_PORT:-5432}:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U delivery -d delivery"]
      interval: 3s
      timeout: 3s
      retries: 20
    restart: unless-stopped
  app:
    build: .
    depends_on:
      db:
        condition: service_healthy
    ports:
      - "${APP_BIND:-127.0.0.1}:8080:8080"
    environment:
      ASPNETCORE_ENVIRONMENT: Development
      ConnectionStrings__Default:
"Host=db;Port=5432;Database=delivery;Username=delivery;Password=$
{POSTGRES_PASSWORD:-delivery}"
      Seed__Demo: "${SEED_DEMO:-true}"
    restart: unless-stopped
volumes:
  pgdata:
```

Назначение инструкций и параметров описано в подразделах 3.6 и 3.7.

Приложение Д

(справочное)

Дополнительные результаты тестирования

Команда запуска: dotnet test OrderDeliveryManagementSystem.sln. Итог: модульные тесты – 41 из 41 пройдены (0,1 с), интеграционные – 236 из 236 пройдены (77 с). Перечень выполненных тестов по классам приведён в таблице Д.1.

Таблица Д.1 – Автоматические тесты по классам

№	Класс	Тестов	Результат
1	OrderModule.MoneyTests	19	Пройдены
2	OrderModule.RulesTests	22	Пройдены
3	Integration.AccountSecurityTests	14	Пройдены
4	Integration.ApiRoleTests	33	Пройдены
5	Integration.ApiValidationTests	15	Пройдены
6	Integration.ApiScenarioTests	9	Пройдены
7	Integration.AssignmentRecoveryTests	14	Пройдены
8	Integration.AssignmentRetryServiceTests	14	Пройдены
9	Integration.AssignmentRetryApiTests	1	Пройдены
10	Integration.ChangeLogTests	8	Пройдены
11	Integration.SessionAndPasswordTests	8	Пройдены
12	Integration.ProductionSeedTests	4	Пройдены
13	Integration.AssignmentAndRetryTests	8	Пройдены
14	Integration.RetryApiTests	1	Пройдены
15	Integration.IdempotencyTests	17	Пройдены
16	Integration.IdempotencyApiTests	2	Пройдены
17	Integration.MigrationTests	10	Пройдены
18	Integration.RegistrationTests	10	Пройдены
19	Integration.ReliabilityTests	10	Пройдены
20	Integration.ScenarioTests	22	Пройдены
21	Integration.StartupConfigTests	21	Пройдены
22	Integration.ValidationTests	15	Пройдены
	Итого	277	Пройдены

Приложение Е
(справочное)
Граф истории Git

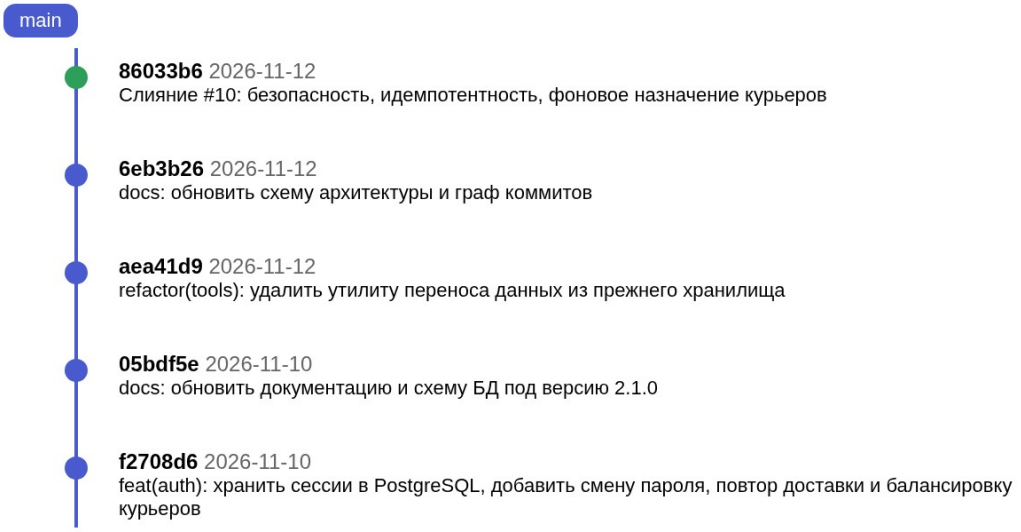


Рисунок Е.1 – Граф коммитов репозитория

Таблица Е.1 – Коммиты репозитория

Хэш	Дата	Сообщение
`f2708d6`	2026-11-10	feat(auth): хранить сессии в PostgreSQL, добавить смену пароля, повтор доставки и балансировку курьеров
`05bdf5e`	2026-11-10	docs: обновить документацию и схему БД под версию 2.1.0
`aea41d9`	2026-11-12	refactor(tools): удалить утилиту переноса данных из прежнего хранилища
`6eb3b26`	2026-11-12	docs: обновить схему архитектуры и граф коммитов
`86033b6`	2026-11-12	Слияние #10: безопасность, идемпотентность, фоновое назначение курьеров

Приложение Ж

(справочное)

Дополнительные листинги кода

Листинг Ж.1 – Служба сессий (src/ApiHost/Auth.cs)

```
/// <summary>Сессии по токену (Bearer). Хранятся в памяти, живут 12 часов.</summary>
public class AuthService
{
    private readonly ConcurrentDictionary<string, (Session s, DateTime exp)>
        _sessions = new();


    public string Start(Session s)
    {
        var token = Convert.ToHexString(RandomNumberGenerator.GetBytes(24));
        _sessions[token] = (s, DateTime.UtcNow.AddHours(12));
        return token;
    }


    public void End(string token) => _sessions.TryRemove(token, out _);


    public static string? Token(HttpRequest r)
    {
        var h = r.Headers.Authorization.ToString();
        return h.StartsWith("Bearer ", StringComparison.OrdinalIgnoreCase) ?
            h[7..].Trim() : null;
    }


    public Session? Get(HttpRequest r)
    {
        var t = Token(r);
        if (t is null || !_sessions.TryGetValue(t, out var v)) return null;
        if (v.exp < DateTime.UtcNow) { _sessions.TryRemove(t, out _); return null; }
        return v.s;
    }
}
```

Листинг Ж.2 – Правила состояний курьеров (src/CourierModule/CourierRules.cs)

```
/// <summary>Правила доступности курьеров: Available, Busy, Offline,
    Blocked.</summary>
public static class CourierRules
{
    public static bool CanAssign(CourierState s) => s == CourierState.Available;


    /// <summary>Проверка ручной смены состояния. Возвращает код ошибки или
        null.</summary>
    public static string? ValidateStateChange(CourierState from, CourierState to,
        bool byOperator)
    {
        if (from == to) return null;
        if (from == CourierState.Busy) return "COURIER_BUSY";
        if (to == CourierState.Busy) return "INVALID_COURIER_STATE";
        if ((to == CourierState.Blocked || from == CourierState.Blocked) &&
            !byOperator) return "FORBIDDEN";
        return null;
    }
}
```

Листинг Ж.3 – Правила заказа (src/OrderModule/OrderRules.cs)

```
/// <summary>Бизнес-правила заказа: состав, редактирование, отмена
    клиентом.</summary>
public static class OrderRules
{
    public const int MaxQuantity = 99;


    /// <summary>Проверка количеств позиций. Возвращает код ошибки или
        null.</summary>
    public static string? ValidateQuantities(IReadOnlyCollection<int>? quantities)
    {
        if (quantities is null || quantities.Count == 0) return "EMPTY_ORDER";
        return quantities.Any(q => q < 1 || q > MaxQuantity) ? "INVALID_QUANTITY" :
            null;
    }


    /// <summary>Заказ можно менять, пока курьер не принял его в работу.</summary>
    public static bool CanEdit(DeliveryStatus s) => s is DeliveryStatus.Created or
        DeliveryStatus.Assigned;


    /// <summary>Клиент может отменить заказ только до передачи в
        доставку.</summary>
    public static bool CanClientCancel(DeliveryStatus s) => s is
        DeliveryStatus.Created or DeliveryStatus.Assigned;


    public static decimal Total(IEnumerable<(decimal Price, int Quantity)> lines) =>
        lines.Sum(l => l.Price * l.Quantity);


    public static OrderStatus StatusFor(DeliveryStatus s) =>
        DeliveryRules.ToOrderStatus(s);
}
```

Листинг Ж.4 – Глобальная обработка ошибок клиента (src/WpfClient/App.xaml.cs)

```
namespace WpfClient;


public partial class App : Application
{
    public static Api Api { get; } = new();


    public App()
    {
        DispatcherUnhandledException += (_, e) =>
        {
            MessageBox.Show(e.Exception.Message, "Ошибка", MessageBoxButton.OK,
                MessageBoxImage.Warning);
            e.Handled = true;
        };
    }
}
```